Diffusion Models and De-Diffusion

Lecture 21
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

12 November, 2026

What this lecture covers

Recap: diffusion with text

Image-conditioned diffusion

Text as an image representation

De-Diffusion

Applications of De-Diffusion

De-Diffusion V2

Summary

Sources, figures and colors

The lecture has two parts. The first recalls the text-conditioned diffusion models of Lecture 20, adds what the trained network estimates, how generation runs and how guidance works, and then shows how a diffusion model is steered by an image such as an edge map or a pose. The second presents De-Diffusion (Wei et al., 2024), which turns a pretrained text-to-image diffusion model around and uses it to train a model that describes images in words, and its successor, the Vision-Language-Vision (VLV) auto-encoder (Zhang et al., 2025).

Pictures taken from a paper or from the course slides say so in their caption. A caption that says "De-Diffusion slides p. N" took its picture from page N of the course slides *De-Diffusion: Diffusion for Training VLMs* (Yuille, Wei and Xiao); "3D-DST slides" are W. Ma's course slides on diffusion models, from a talk on 3D-DST, his method for generating images with 3D annotations (Ma et al., 2024). In the block diagrams drawn for these notes, blue marks the encoder and the text path, orange the time step and the zero convolutions, purple the denoising network and teal the decoder and the trainable copy. In the toy plots, blue points are samples and grey circles the clusters of the data. Other pictures use colors only to tell elements apart.

Denoising diffusion models consist of two processes

This section recalls the diffusion models of Lecture 20 (Sohl-Dickstein et al., 2015; Ho et al., 2020), in the notation used there, and adds what the rest of this lecture needs: what the trained network estimates, how generation runs, and how a prompt steers it.

The *forward process* is fixed: it adds a little Gaussian noise at each of T steps, $\mathbf{x}_0 \to \cdots \to \mathbf{x}_T$, until $\mathbf{x}_T$ is pure noise. With the noise schedule $\beta_1, \ldots, \beta_T$ (Ho et al. use $T = 1000$ and a linear rise from 0.0001 to 0.02), $\mathbf{x}_t$ can be drawn from $\mathbf{x}_0$ in one step:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s).$$

The *reverse process* is learnt. A U-Net (Ronneberger et al., 2015) $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$ looks at a noisy image and the step and predicts the noise that was added, trained with the squared loss

$$L(\theta) = \mathbb{E}_{\mathbf{x_0}, t, \boldsymbol{\varepsilon}} \left\| \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon},\ t\big) \right\|^2.$$

Denoising diffusion models consist of two processes

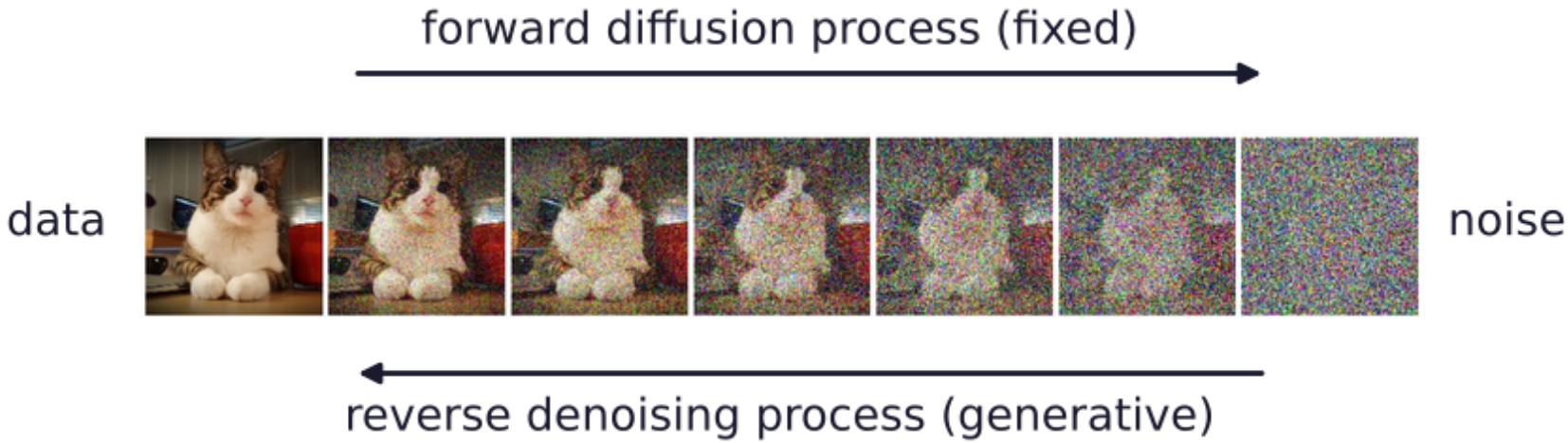


Figure 1: The two processes on one photograph (from the CVPR 2022 tutorial of Kreis, Gao and Vahdat).

Predicting the noise is predicting the clean image

Given $\mathbf{x}_t$, the noise and the clean image determine each other: $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon})/\sqrt{\bar{\alpha}_t}$. So a noise prediction $\hat{\boldsymbol{\varepsilon}}$ is also a prediction of the clean image,

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\varepsilon}}}{\sqrt{\bar{\alpha}_t}}.$$

Which $\hat{\boldsymbol{\varepsilon}}$ minimizes the loss? For a fixed $\mathbf{x}_t$, the expected squared error $\mathbb{E}\|\boldsymbol{\varepsilon} - \hat{\boldsymbol{\varepsilon}}\|^2$ is smallest at the mean, $\hat{\boldsymbol{\varepsilon}} = \mathbb{E}[\boldsymbol{\varepsilon} \mid \mathbf{x}_t]$. This is the rule of Bayes decision theory for a squared loss: the best estimate is the posterior mean. The trained network therefore estimates the posterior mean of the clean image given the noisy one, $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t]$, and the prior that this posterior uses is the distribution of the training images.

At large t the noisy image says little, and the posterior mean is a blurry average of many plausible images; at small t it is close to one image. Generation uses this guess, and the guess is also the gradient of a log density; both facts are used for generation and guidance.

Recap: Stable Diffusion denoises a latent given the prompt

Stable Diffusion is a latent diffusion model (Rombach et al., 2022). An image autoencoder, trained first, has an encoder $\mathcal{E}$ that maps a $512 \times 512 \times 3$ image to a $64 \times 64 \times 4$ *latent* and a decoder $\mathcal{D}$ that maps a latent back to an image. The diffusion model works on latents: all the formulas above hold with $\mathbf{z}_t$ in place of $\mathbf{x}_t$, and a generated latent $\mathbf{z}_0$ is decoded, $\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z}_0)$.

The prompt y is a sequence of words. A frozen text encoder τ turns it into one feature vector per token; Stable Diffusion v1 uses the text encoder of CLIP, Contrastive Language-Image Pre-training (Radford et al., 2021). The network becomes $\varepsilon_\theta(\mathbf{z}_t, t, \tau(y))$ and is trained on image-caption pairs with the same loss; it now estimates $\mathbb{E}[\mathbf{z}_0 \mid \mathbf{z}_t, y]$, and sampling with it draws from $p(\mathbf{z}_0 \mid y)$.

The text enters every block of the U-Net through cross-attention (Vaswani et al., 2017), $\mathrm{softmax}(QK^\mathsf{T}/\sqrt{d})\,V$: the queries $Q = W_Q\,\varphi(\mathbf{z}_t)$ come from the U-Net features $\varphi(\mathbf{z}_t)$, one vector per position, and the keys $K = W_K\,\tau(y)$ and values $V = W_V\,\tau(y)$ from the text. Each position takes in the features of the words that describe it.

Recap: Stable Diffusion denoises a latent given the prompt

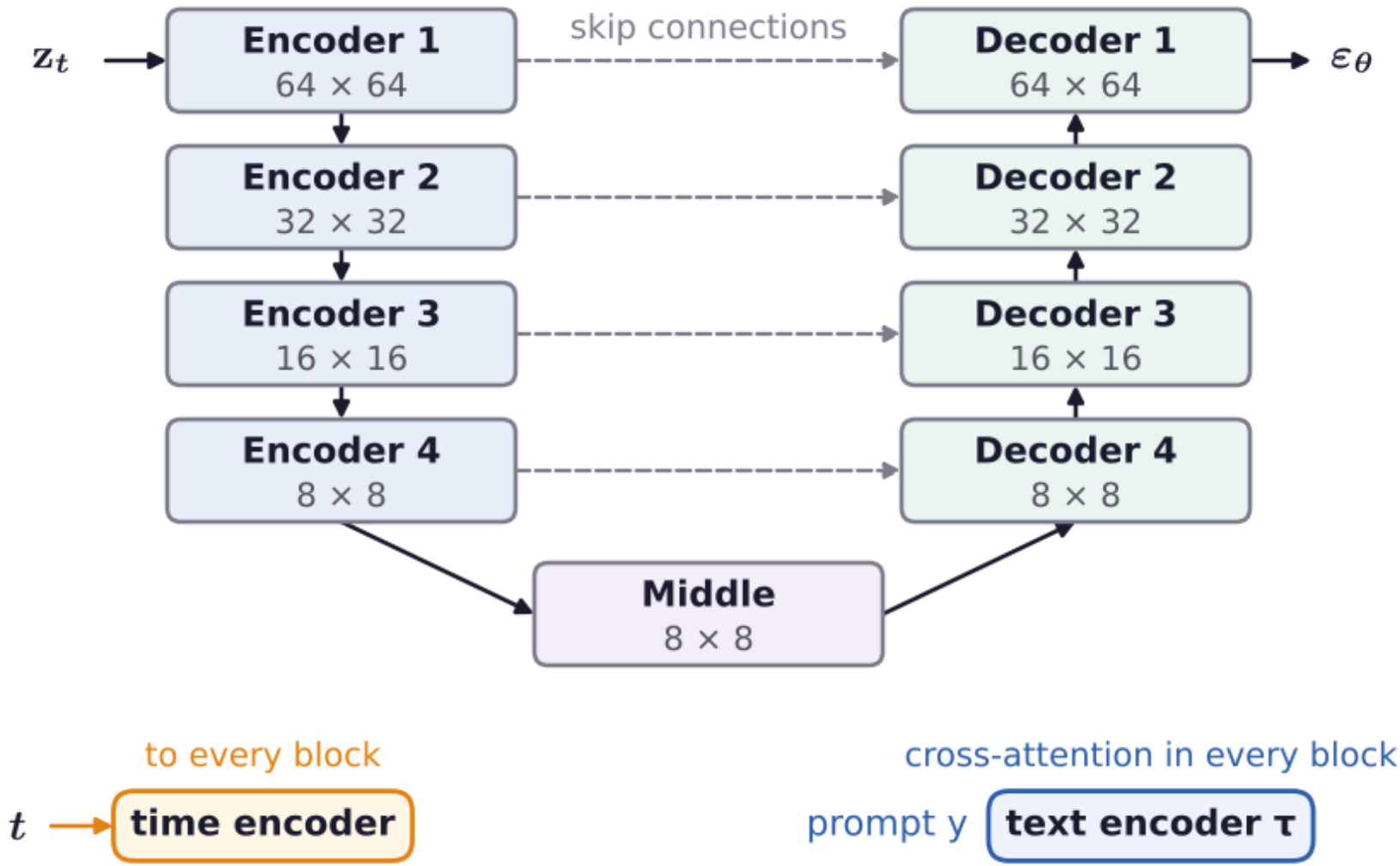


Figure 2: The U-Net of Stable Diffusion with the text path (after Zhang et al., 2023, Figure 3): the time step and, through cross-attention, the text features $\tau(y)$ enter every block.

Generation denoises step by step, from noise to data

Generation starts from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and applies, for $t = T, \dots, 1$,

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\,\mathbf{n}, \qquad \mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

with no noise at the last step. The fresh noise $\sigma_t \mathbf{n}$ keeps the chain random, so different runs give different images; Ho et al. take $\sigma_t^2 = \beta_t$. Throughout, $\boldsymbol{\varepsilon}_\theta$ is the network and $\boldsymbol{\varepsilon}$ the noise. Written this way the step is hard to read; the frames after the toy example rewrite it in two ways that show what it does.

The optimal noise predictor of a Gaussian mixture

The toy example of the figure uses a distribution for which the best possible network can be written down. Let the data be a mixture of K Gaussians (here K counts clusters), $\mathbf{x}_0 \sim \sum_k a_k \mathcal{N}(\boldsymbol{\mu}_k, s^2 \mathbf{I})$ with weights a_k. In the closed form, a noisy point from cluster k is Gaussian, $\mathbf{x}_t \mid k \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\boldsymbol{\mu}_k,\ v_t \mathbf{I})$ with $v_t = \bar{\alpha}_t s^2 + 1 - \bar{\alpha}_t$. Bayes' rule gives the responsibility of cluster k for $\mathbf{x}_t$,

$$r_k(\mathbf{x}_t) = \frac{a_k\, \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\boldsymbol{\mu}_k, v_t \mathbf{I})}{\sum_j a_j\, \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\boldsymbol{\mu}_j, v_t \mathbf{I})},$$

and within a cluster the posterior mean of $\mathbf{x}_0$ is that of two jointly Gaussian variables, $\boldsymbol{\mu}_k + (\sqrt{\bar{\alpha}_t} s^2 / v_t)(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\boldsymbol{\mu}_k)$. Averaging over the clusters with weights r_k gives $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t]$, and the optimal predictor is $\varepsilon^*_\theta(\mathbf{x}_t, t) = (\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t])/\sqrt{1-\bar{\alpha}_t}$.

In the figure, $K = 8$ equal clusters with $s = 0.12$ sit on a circle of radius 2; 2000 chains follow the sampling step with ε^*_θ. They reach the ring by $t = 150$ and have chosen a cluster by $t = 60$; at $t = 0$, 99.9% lie within $4s$ of a center.

Generation denoises step by step, from noise to data

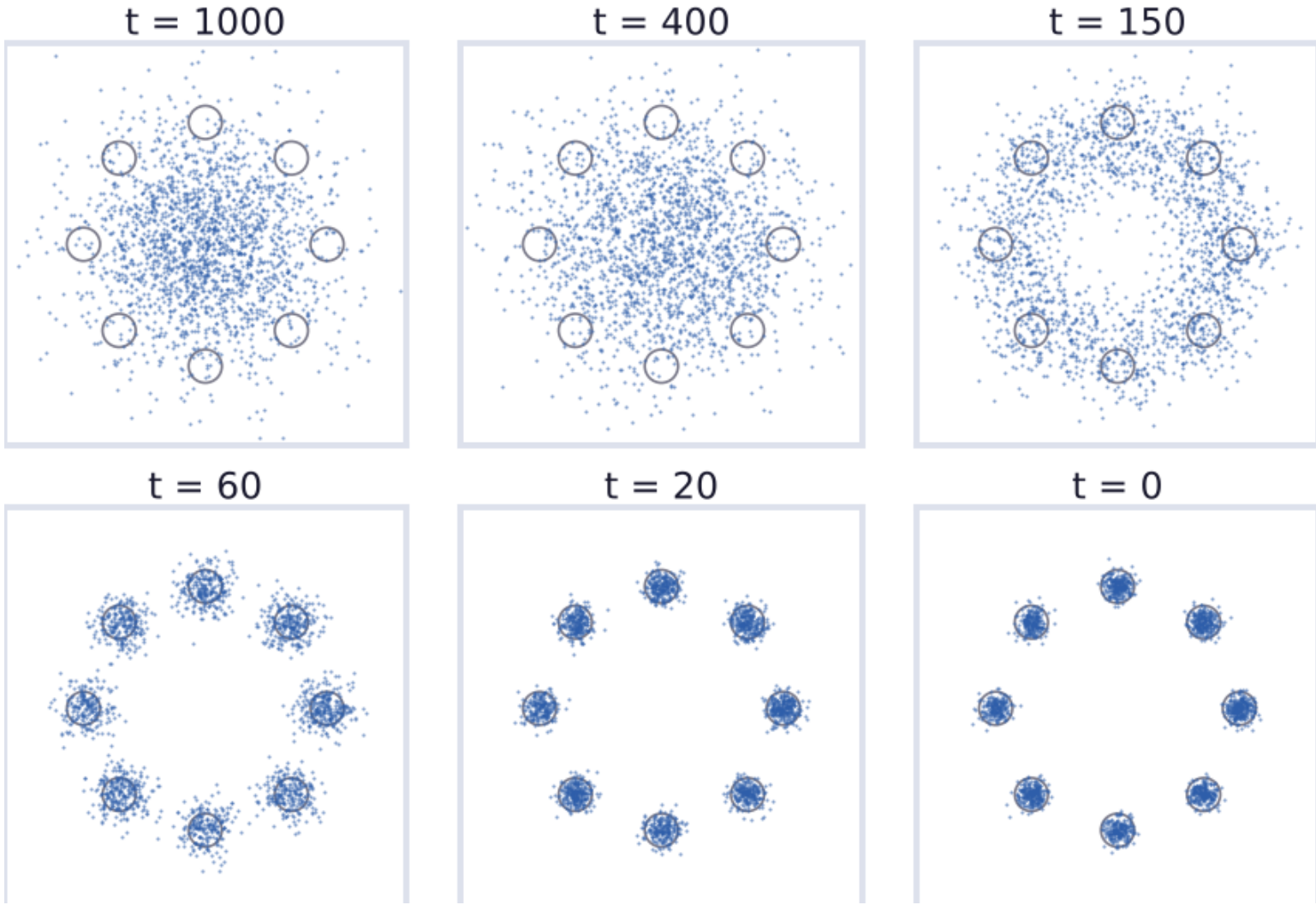


Figure 3: Reverse diffusion with the exact optimal predictor: each blue dot is one chain at step t; grey circles mark the eight clusters of the data.

Each step moves a little towards the predicted clean image

Write the network's guess of the clean image,
$\hat{\mathbf{x}}_0(\mathbf{x}_t) = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t))/\sqrt{\bar{\alpha}_t}$, an estimate of $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t]$.
Substituting it into the sampling step gives

$$\mathbf{x}_{t-1} = a_t\,\hat{\mathbf{x}}_0(\mathbf{x}_t) + b_t\,\mathbf{x}_t + \sigma_t\,\mathbf{n}, \qquad a_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}, \quad b_t = \frac{\sqrt{1-\beta_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t},$$

the weights of the exact step back if the clean image were known, the mean of $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ (Ho et al., 2020, eq. 7). The sampler treats its current guess as the truth, takes the exact step for that case and adds fresh noise; the guess is made again at every step, so early mistakes can be corrected. The weight on the guess is tiny at first ($a_t \approx 0.0001$ at $t = 1000$, 0.01 at $t = 200$, 0.15 at $t = 10$) and $a_1 = 1$: generation is a thousand small corrections, and the last step outputs the guess.

In the toy example (figure) the guesses of all chains sit near the origin at $t = 600$: the posterior mean of eight equally likely clusters is close to their average. By $t \approx 220$ they reach the ring of radius 2, but most lie between clusters. The choice of a cluster comes later and quickly: half of the chains have a responsibility r_k above 0.9 at $t \approx 148$, 90% at $t \approx 97$. The coarse structure is decided first and the details last; Ho et al. (2020) report the same order for images, where the large-scale features of the predicted image appear first.

Each step moves a little towards the predicted clean image

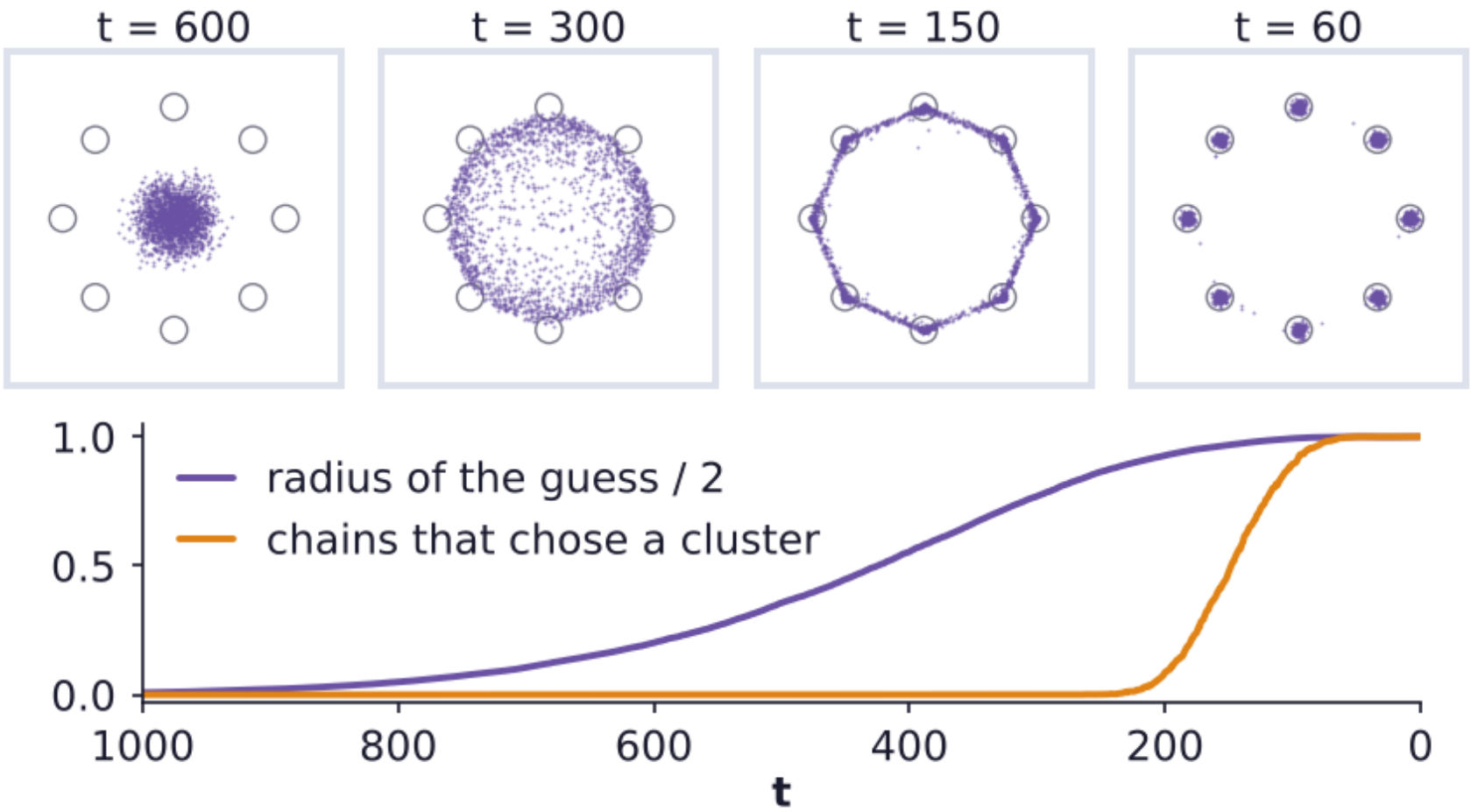


Figure 4: Top: the guess $\hat{\mathbf{x}}_0$ of the 2000 chains at four steps (grey circles: the clusters). Bottom, against t (generation runs left to right): the mean distance of the guess from the origin over the ring radius 2 (purple), and the fraction of chains with a responsibility above 0.9 (orange).

Predicted noise points down the log-density of noisy images

Let p_t be the density of the noisy images $\mathbf{x}_t$: the training images, shrunk by $\sqrt{\bar{\alpha}_t}$ and blurred by Gaussian noise of variance $1 - \bar{\alpha}_t$. Tweedie's formula says that the posterior mean points uphill on this density:

$$\nabla \log p_t(\mathbf{x}_t) = \frac{\sqrt{\bar{\alpha}_t}\,\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t] - \mathbf{x}_t}{1 - \bar{\alpha}_t}, \quad \text{so} \quad \mathbb{E}[\varepsilon \mid \mathbf{x}_t] = -\sqrt{1 - \bar{\alpha}_t}\,\nabla \log p_t.$$

A short argument shows why. Write $p_t(\mathbf{x}_t) = \int p_0(\mathbf{x}_0)\,\mathcal{N}_t(\mathbf{x}_t \mid \mathbf{x}_0)\,d\mathbf{x}_0$, with $\mathcal{N}_t(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$. The gradient of $\mathcal{N}_t$ is $\mathcal{N}_t$ times $(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 - \mathbf{x}_t)/(1 - \bar{\alpha}_t)$; dividing by p_t turns the integral into an average over the posterior $p(\mathbf{x}_0 \mid \mathbf{x}_t)$. So the trained network, which estimates $\mathbb{E}[\varepsilon \mid \mathbf{x}_t]$, also estimates the gradient of $\log p_t$ up to the factor $-\sqrt{1 - \bar{\alpha}_t}$: the predicted noise points downhill, and the step, which subtracts it, goes uphill.

Predicted noise points down the log-density of noisy images

For small β_t, $1/\sqrt{1-\beta_t} \approx 1 + \beta_t/2$, and the sampling step becomes

$$\mathbf{x}_{t-1} \approx \mathbf{x}_t + \beta_t\big(\nabla \log p_t(\mathbf{x}_t) + \tfrac{1}{2}\mathbf{x}_t\big) + \sqrt{\beta_t}\,\mathbf{n}.$$

This is a Langevin step: a gradient step on $\log p_t$ plus noise of matched size (the term $\frac{1}{2}\mathbf{x}_t$ undoes the shrinking of the forward step). A Langevin sampler run on p_0 alone would get stuck: far from the data the gradient is tiny, and between separated hills there is no path. Diffusion avoids this by changing the density as it goes. At large t, p_t is one broad hill (figure, $t = 400$) that every chain climbs easily; as t falls it splits into separate hills, and each chain is already close to one of them. The noise level plays the role of the temperature in simulated annealing.

The gradient form also shows how a prompt y acts. By Bayes' rule, $p(\mathbf{x}_t \mid y) \propto p(\mathbf{x}_t)\, p(y \mid \mathbf{x}_t)$, so

$$\nabla \log p_t(\mathbf{x}_t \mid y) = \nabla \log p_t(\mathbf{x}_t) + \nabla \log p_t(y \mid \mathbf{x}_t)\,:$$

the gradient of the prior, learnt from all images, plus the gradient of a likelihood, the log probability that a noisy image fits the prompt. The prompt does not replace the prior; it adds a pull towards images that match it. Guidance (next) scales this term, and ControlNet adds a third.

Predicted noise points down the log-density of noisy images

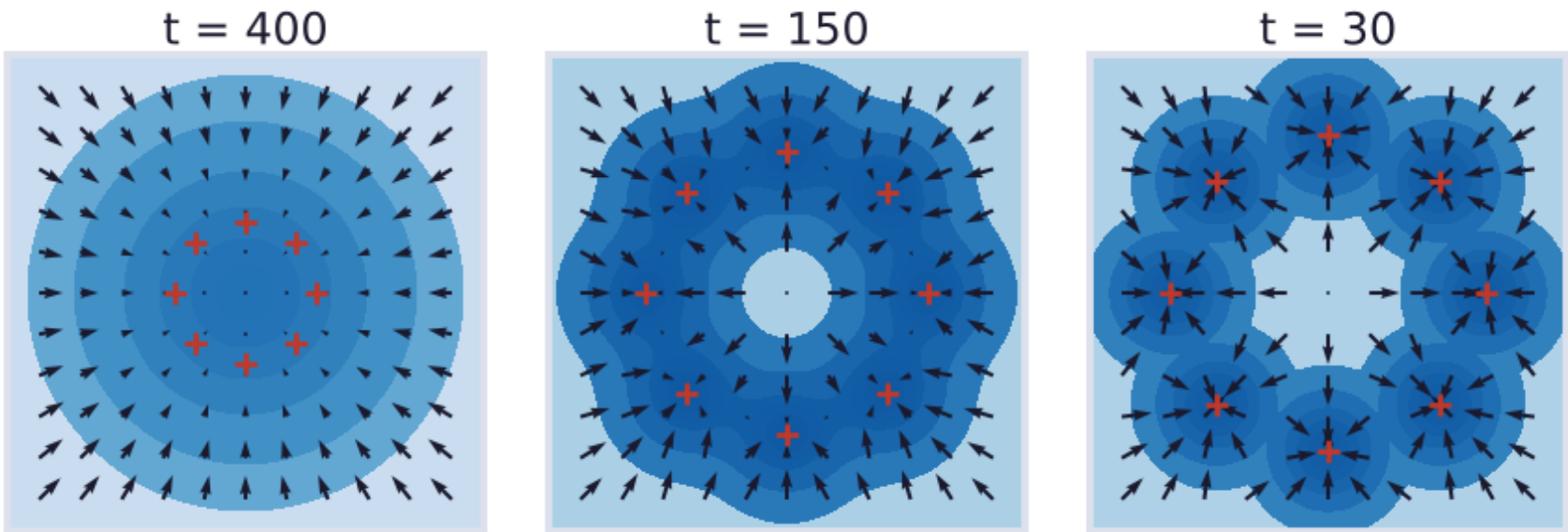


Figure 5: The toy's $\log p_t$ (darker blue: higher) at $t = 400$, 150 and 30, with its gradient (arrows; long arrows cut to a fixed length). Red crosses: the cluster centers shrunk by $\sqrt{\bar{\alpha}_t}$, the tops of the hills.

Classifier-free guidance strengthens the text condition

Sampled with the conditional network alone, the images often match the prompt only loosely. Classifier-free guidance (Ho and Salimans, 2021) fixes this with one network that has learnt two predictions: during training the prompt is replaced at random (for 10–20% of the examples) by the empty prompt $\varnothing$, so the network learns both $\varepsilon_\theta(\mathbf{z}_t, t, y)$ and the unconditional $\varepsilon_\theta(\mathbf{z}_t, t, \varnothing)$. At sampling time the two are combined,

$$\tilde{\varepsilon}_\theta(\mathbf{z}_t, t, y) = \varepsilon_\theta(\mathbf{z}_t, t, \varnothing) + w\,\big(\varepsilon_\theta(\mathbf{z}_t, t, y) - \varepsilon_\theta(\mathbf{z}_t, t, \varnothing)\big),$$

and $\tilde{\varepsilon}_\theta$ is used in the sampling step. The *guidance scale* w sets how far to move along the direction that the prompt adds: $w = 1$ is plain conditional sampling, $w = 0$ ignores the prompt, and $w > 1$ goes beyond the conditional prediction. Typical values are 3 to 9.

Larger w follows the prompt more closely but gives less diverse images.

Guidance sharpens the conditional distribution

By Tweedie's formula, predicted noise is a scaled gradient of a log density, $\varepsilon_\theta \approx -\sqrt{1-\bar{\alpha}_t}\,\nabla \log p_t$. Applied to the guided prediction,

$$\begin{aligned}\nabla \log \tilde{p}_t(\mathbf{z}_t \mid y) &= \nabla \log p_t(\mathbf{z}_t) + w\big(\nabla \log p_t(\mathbf{z}_t \mid y) - \nabla \log p_t(\mathbf{z}_t)\big) \\ &= \nabla \log p_t(\mathbf{z}_t) + w\,\nabla \log p_t(y \mid \mathbf{z}_t),\end{aligned}$$

by Bayes' rule, $p(\mathbf{z} \mid y) = p(y \mid \mathbf{z})\,p(\mathbf{z})/p(y)$. So guidance samples, approximately, from

$$\tilde{p}(\mathbf{z} \mid y) \;\propto\; p(\mathbf{z})\,p(y \mid \mathbf{z})^w :$$

the prior of images times the likelihood that the image shows the prompt, raised to the power w. For $w > 1$ the likelihood is sharpened, and images that are only somewhat consistent with the prompt lose probability. The method is "classifier-free" because $p(y \mid \mathbf{z})$, a classifier of noisy images, is never trained: it is implied by the conditional and unconditional networks through Bayes' rule. In the toy example (three Gaussian classes), 12% of the class-1 samples lie nearer another center at $w = 1$, almost none from $w = 2$ on, and for large w they are pushed beyond their own center.

Classifier-free guidance strengthens the text condition

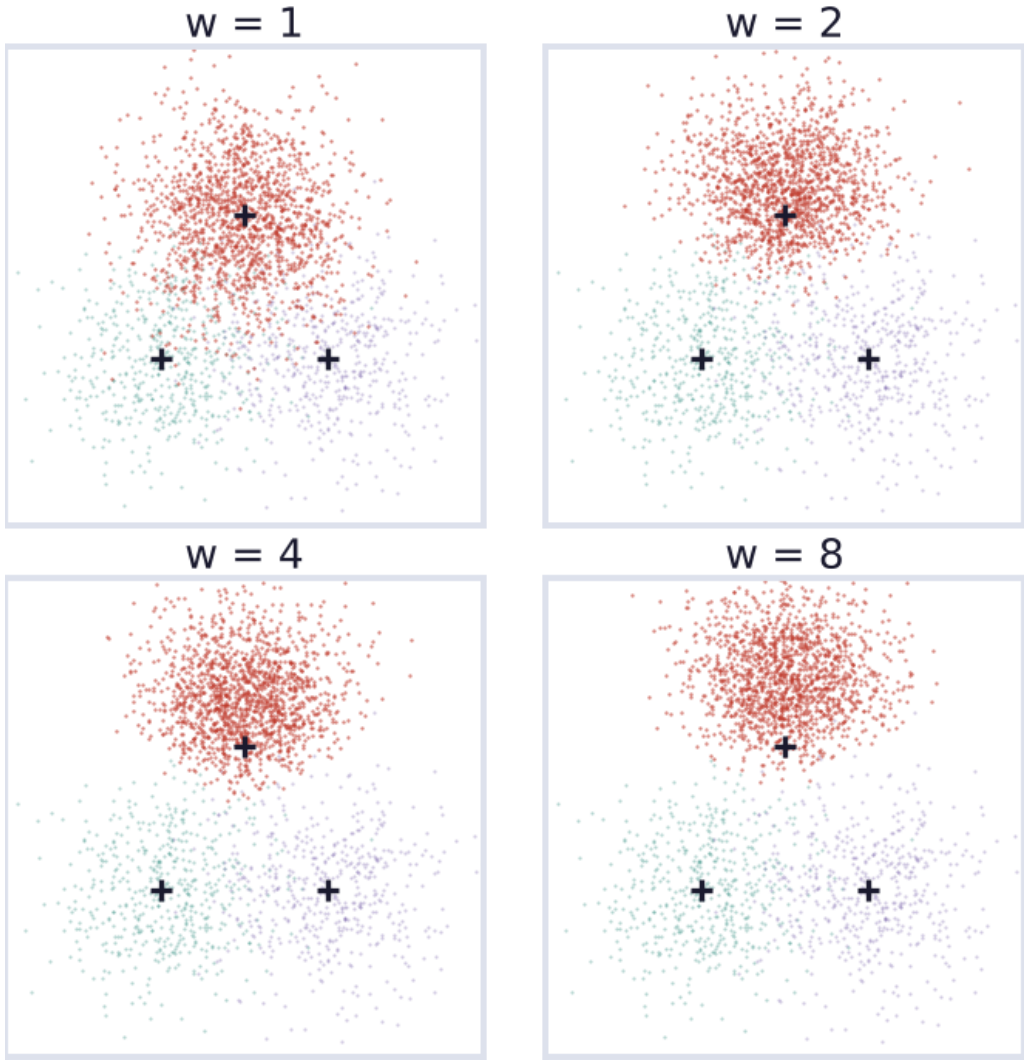


Figure 6: Guided samples of class 1 (red, top) at four scales w; other classes in teal and purple; crosses mark the centers. Each class is a Gaussian (standard deviation 0.9, centers on a circle of radius 1.6). From $w = 4$ the red cloud sits beyond its center.

Diffusion models also keep a character and edit images

A *character reference* ("cref", an option of the commercial generator Midjourney) takes a reference picture of a person and makes the model draw that person in new scenes. In *prompt-based editing* (Prompt-to-Prompt; Hertz et al., 2023) one word of the prompt is replaced (handwritten in the figure) or its weight raised or lowered (arrows), and only the part described by that word changes.

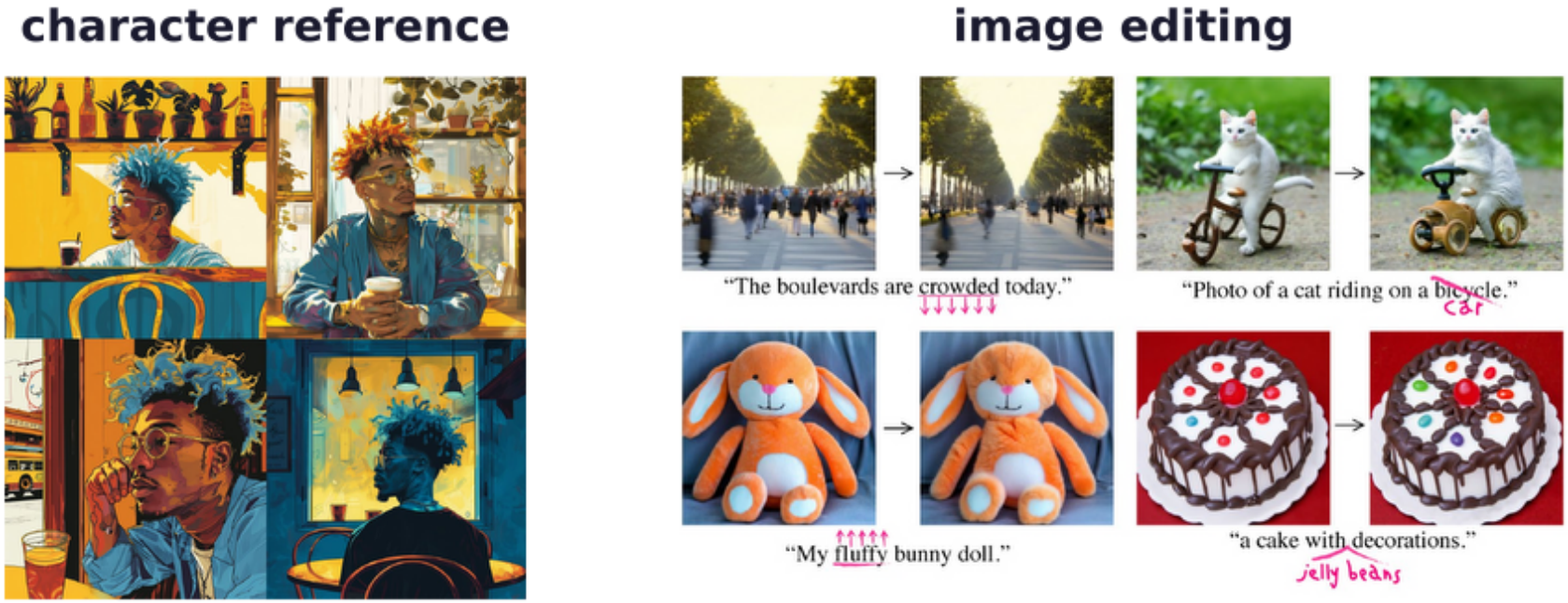


Figure 7: One character in four scenes, and four prompt edits (Hertz et al., 2023).

ControlNet adds visual conditioning with zero convolutions

Text fixes what is in an image but only loosely where. ControlNet (Zhang et al., 2023) adds visual conditioning to text-to-image diffusion models: an image $\mathbf{c}$ such as an edge map, a pose skeleton or a depth map fixes the layout. The base text-to-image model is fixed, and various visual conditionings are achieved by training only an adapter on top of the large pretrained model.

Take one block $\mathcal{F}(\cdot;\Theta)$ of the pretrained network, with input features $\mathbf{x}$ and output $\mathbf{y} = \mathcal{F}(\mathbf{x};\Theta)$. ControlNet locks Θ, makes a trainable copy Θ_c of the same block, and joins the copy to the locked block by two zero convolutions $\mathcal{Z}$:

$$\mathbf{y}_c = \mathcal{F}(\mathbf{x};\Theta) + \mathcal{Z}\big(\mathcal{F}\big(\mathbf{x} + \mathcal{Z}(\mathbf{c};\Theta_{z1});\Theta_c\big);\Theta_{z2}\big).$$

A *zero convolution* is a 1×1 convolution whose weights and bias start at zero. At the start of training both $\mathcal{Z}$ output zero, so $\mathbf{y}_c = \mathbf{y}$: the model is exactly the pretrained model. Such a layer still learns; the reason follows the overall layout.

ControlNet adds visual conditioning with zero convolutions

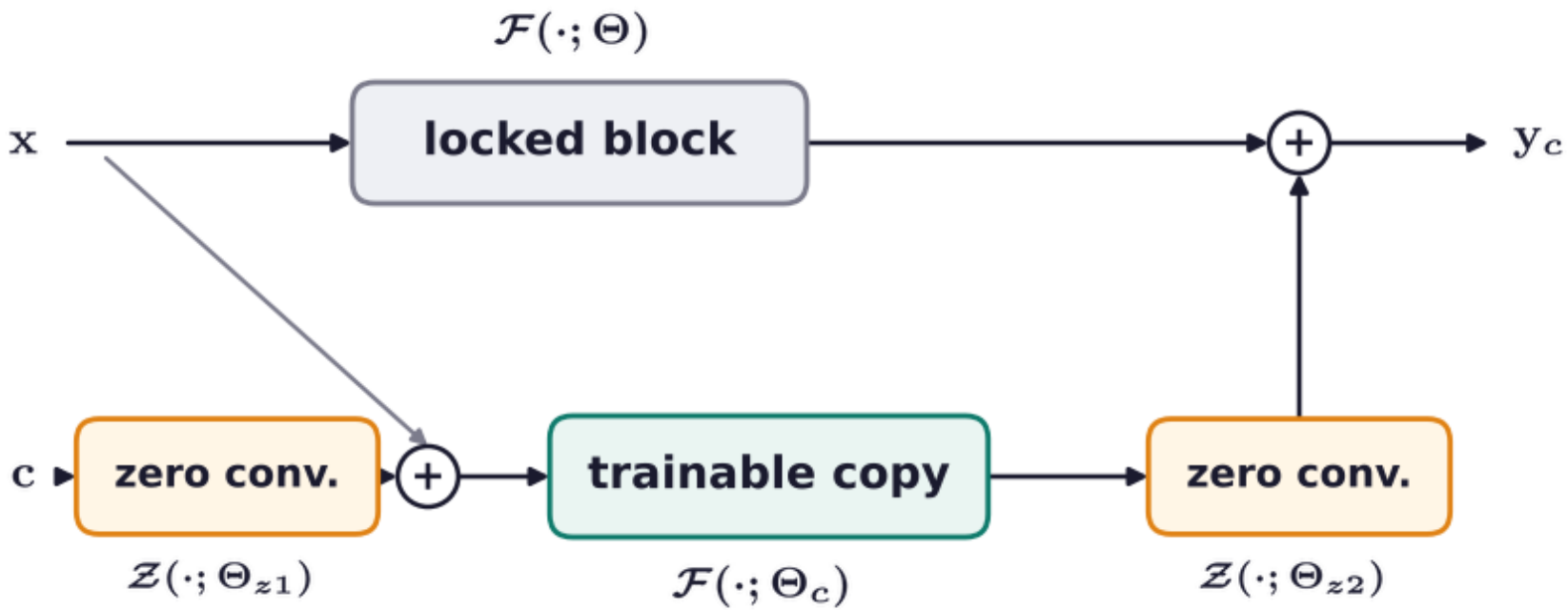


Figure 8: One ControlNet block (after Zhang et al., 2023, Figure 2). Top: the locked block of the pretrained model. Bottom: the condition $\mathbf{c}$ passes a zero convolution and is added to the input $\mathbf{x}$; the trainable copy processes the sum; a second zero convolution adds its output to the locked block's output, giving $\mathbf{y_c}$.

ControlNet attaches a trainable copy to every encoder block

For Stable Diffusion, ControlNet copies the twelve encoder blocks and the middle block of the U-Net (three blocks at each of the four resolutions 64×64 to 8×8, plus the middle). The outputs of the copies are added, each through its own zero convolution, to the twelve skip connections and to the middle block of the locked U-Net; the decoder of the U-Net is untouched. The prompt and the time step enter the copy as they enter the original.

The condition image, at 512×512, has to match the 64×64 latent. A small convolutional network (ReLU, rectified linear units, between the layers; the released code downsamples by 8) maps it to the latent's resolution before the first zero convolution. Training uses pairs of images and their conditions, for example photographs and the edge maps computed from them, with the usual denoising loss; one ControlNet is trained for each kind of condition. The copy has about as many parameters as the encoder, but the base model's weights, learnt on billions of images, are reused.

Zero convolutions start at the base model yet still learn

A zero convolution is a 1×1 convolution, so at each pixel it is a linear map, $\mathcal{Z}(\mathbf{h}) = W\mathbf{h} + \mathbf{b}$, from the copy's features $\mathbf{h}$ to a vector of the size of the locked block's output; W and $\mathbf{b}$ start at zero, and the output is added to the block, $\mathbf{y}_c = \mathbf{y} + \mathcal{Z}(\mathbf{h})$. For a loss L the chain rule gives

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{y}_c}\,\mathbf{h}^{\mathsf{T}}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}_c}, \qquad \frac{\partial L}{\partial \mathbf{h}} = W^{\mathsf{T}}\frac{\partial L}{\partial \mathbf{y}_c}.$$

At the first step $W = 0$, so $\partial L/\partial \mathbf{h} = 0$: no gradient reaches the trainable copy yet. But $\partial L/\partial W$ is not zero, because the copy's features $\mathbf{h}$ are not zero (the copy is a trained block fed with real features) and neither is the gradient of the loss. The first update makes W nonzero, and from then on gradients flow into the copy and into the first zero convolution.

So training starts from a model that already makes good images, undisturbed by randomly initialized layers, and the copy's contribution grows gradually from zero. The paper reports that with standard, randomly initialized convolutions instead, performance drops to about that of a lighter variant, ControlNet-lite. The same idea appears elsewhere in deep learning: setting the last layer of a residual branch to zero makes a deep network start as the identity.

Edges and human poses control the generated image

Two conditions illustrate the idea. The first is a Canny edge map: the output of the Canny edge detector, a binary image of the intensity edges of a photograph (here a stag). The second is a human pose: a skeleton of colored line segments joining the detected joints of a person. With no prompt ("default"), ControlNet generates images whose edges or pose match the condition and fills in plausible content. With a prompt, the content follows the text and the layout follows the condition: the stag becomes a fairy-tale deer with golden antlers, the posed man a chef in a kitchen or a statue of Lincoln.

Edges and human poses control the generated image



Figure 9: ControlNet's Figure 1 (Zhang et al., 2023). Each row: the condition (edges, pose), two images without a prompt (default), two with the first prompt and one with the second.

The prompt and the control image guide the same denoiser

A ControlNet model has two conditions, the prompt y and the control image $\mathbf{c}$, and they enter in different places (figure). The prompt passes the frozen text encoder τ and enters the blocks of the locked U-Net, and of the trainable copy, by cross-attention, as in Stable Diffusion. The control image passes a small convolutional network (four layers, from 512×512 to the latent's 64×64) and the trainable copy, and its features are added, through zero convolutions, to the twelve skip connections and the middle block. The text says mainly what is in the image; the control, through the skip connections that carry the layout, says where.

Bayes' rule for two conditions shows how they combine:

$$\nabla \log p(\mathbf{z} \mid y, \mathbf{c}) = \underbrace{\nabla \log p(\mathbf{z})}_{\text{prior}} + \underbrace{\nabla \log p(y \mid \mathbf{z})}_{\text{text}} + \underbrace{\nabla \log p(\mathbf{c} \mid \mathbf{z}, y)}_{\text{control}}.$$

Stable Diffusion estimates the first two terms; the change that the ControlNet makes to the noise prediction can be read as an estimate of the third, the gradient of the log probability that the image has these edges or this pose. The paper does not state this reading (the copy adds features rather than a term of the noise prediction); it does explain two of its design choices.

The prompt and the control image guide the same denoiser

First, prompt dropout: during training half of the prompts are replaced by the empty prompt. For those examples the only way to lower the loss is to read the meaning of the condition (this pose is a person, these edges a deer); the paper says this helps ControlNet recognize the semantics of the condition directly. It tests four kinds of prompt: none; an insufficient one (the default "a high-quality, detailed, and professional image"); a conflicting one, whose meaning differs from the condition's; and a full description ("a nice house"). ControlNet gives good images in all four, while ControlNet-lite fails with no prompt or the insufficient one.

Second, composition: several ControlNets (say pose and depth) are used together by adding their outputs, with no extra weighting. In the reading above this adds the log-likelihood gradients of the two controls, exact when they are independent given the image and the prompt.

With guidance, a control added to both branches cancels in the difference that guidance amplifies; added only to the conditional branch, it is multiplied by w and is too strong. The paper adds it to the conditional branch and rescales each connection by a weight that depends on its resolution.

The prompt and the control image guide the same denoiser

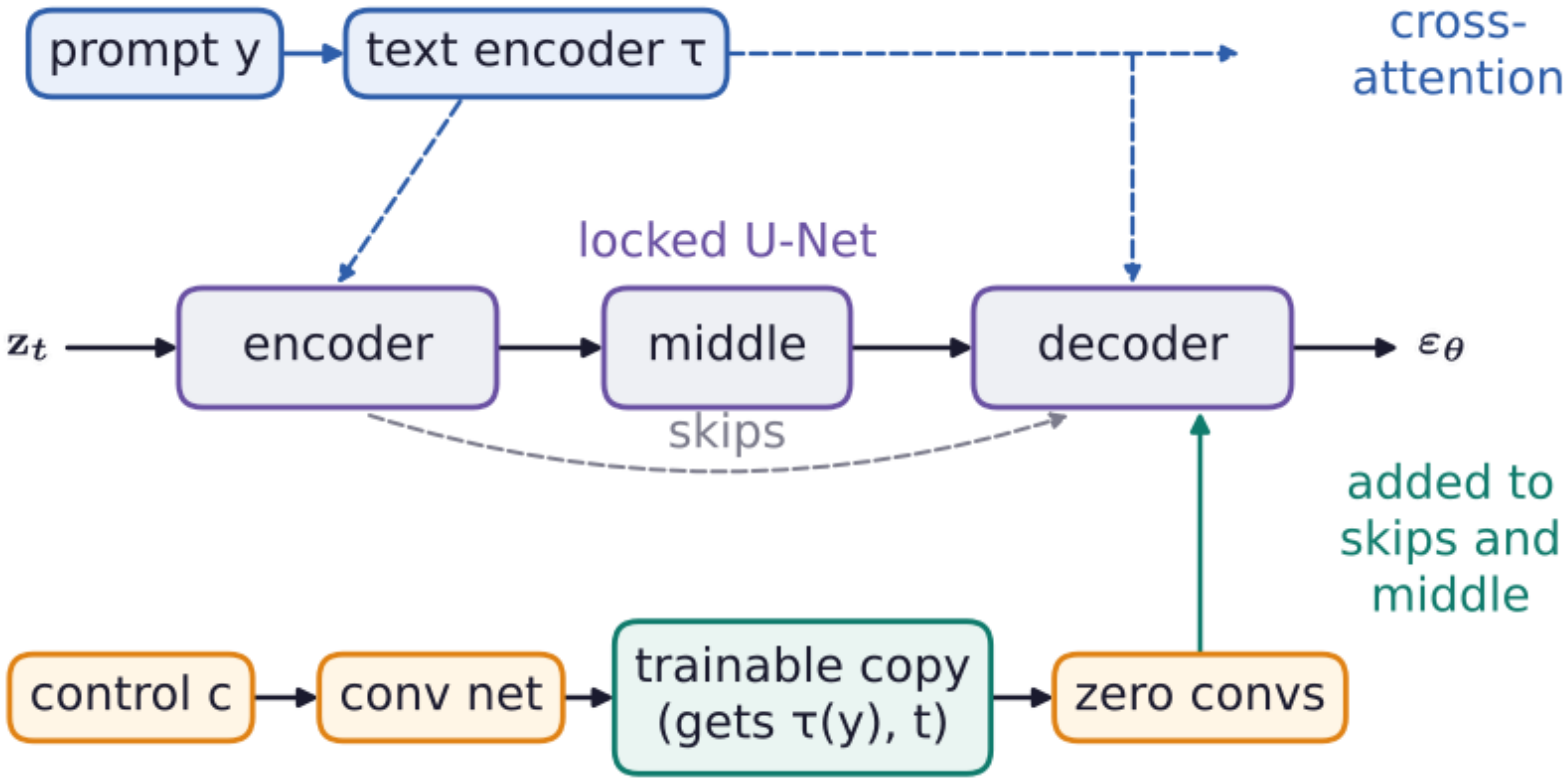


Figure 10: Where the two conditions enter (drawn from Zhang et al., 2023, Figure 3 and its text). Blue dashed: the text features $\tau(y)$ enter the locked U-Net and the trainable copy by cross-attention. Orange and teal: the control $\mathbf{c}$ passes a small convolutional network and the copy; zero convolutions add its features to the skip connections and the middle block. Grey dashed: the skip connections of the U-Net.

ControlNet training converges suddenly

One experiment of the ControlNet paper tests a ControlNet, after more and more training steps, on the same apple sketch and the same prompt, "apple" plus the default "a detailed high-quality professional image" (guidance scale 9.0, learning rate 10^{-5}, AdamW, a variant of the Adam gradient optimizer). For about six thousand steps the output ignores the sketch: a colorful screen, a phone, apples in a landscape, an apple next to a logo. Between 6100 and 6133 steps it suddenly follows the sketch, and stays so.

The authors call this sudden convergence and report it without a full explanation of why it is sudden. The zero convolutions explain why the output stays a good image all along: the trainable copy adds its contribution slowly from zero, so the pretrained model keeps generating high-quality images while the copy learns, and the condition takes over once the copy has learnt it.

ControlNet training converges suddenly

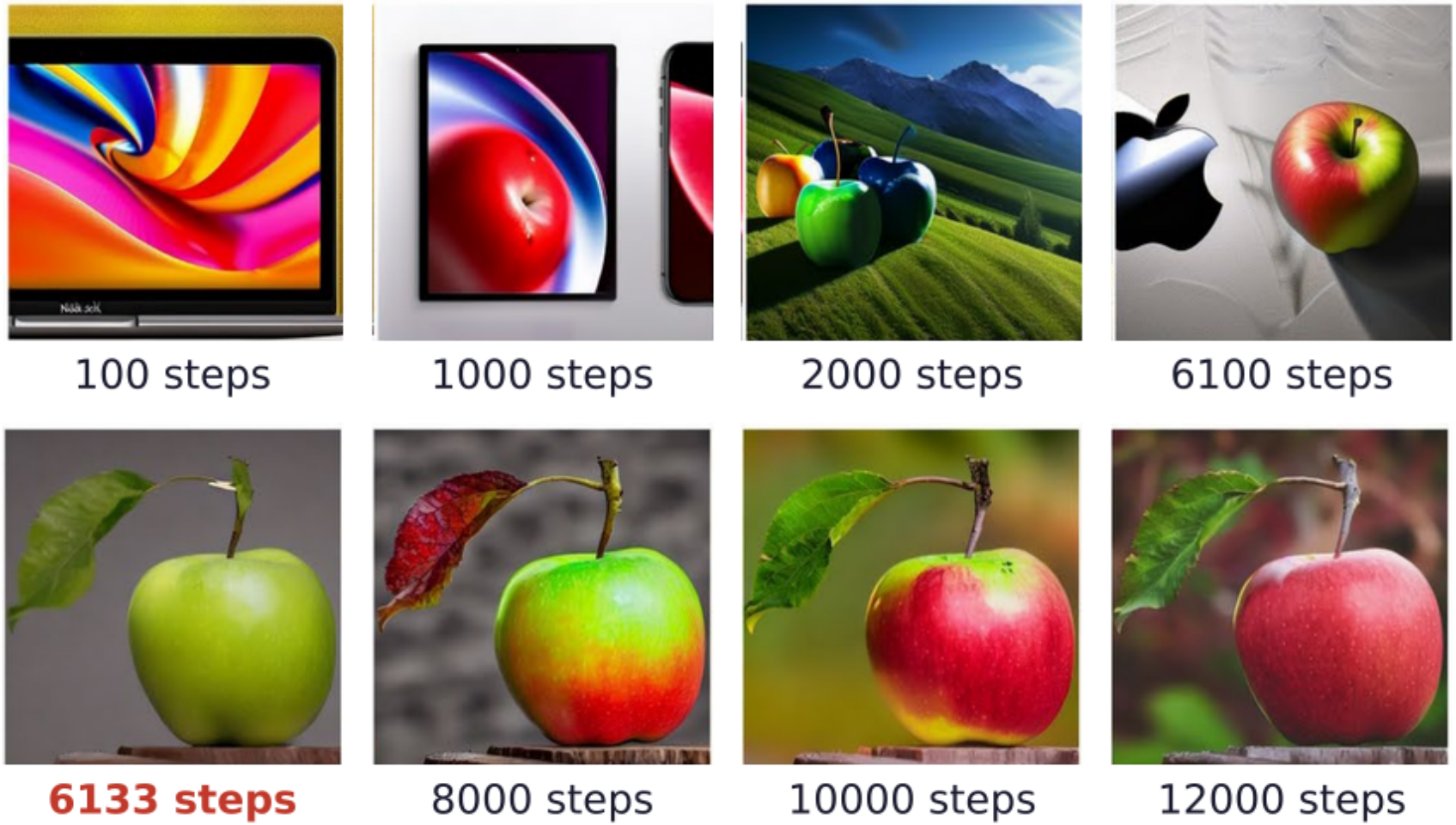


Figure 11: Outputs for the same sketch and prompt after 100 to 12000 training steps (Zhang et al., 2023, Figure 4). The output first follows the sketch at 6133 steps (red). The sketch itself is not shown here; it is a line drawing of one apple.

LooseControl conditions on 3D boxes instead of exact depth

ControlNet's depth condition needs an exact depth map: the distance to the camera at every pixel. To get a room with furniture, a user would have to supply the depth of every wall, table and vase, which is about as hard as making the image. LooseControl (Bhat et al., 2024) lifts this requirement. Its *3D box control* takes the depth map of a few rough 3D boxes, one per main object, and the prompt; the model fills each box with an object of the right kind and adds whatever else the scene needs. A second form, scene boundary control, gives only the walls and floor. LooseControl keeps ControlNet's architecture and fine-tunes it on such loose depth maps with LoRA (low-rank adaptation: a small trainable low-rank correction added to each weight matrix).

LooseControl conditions on 3D boxes instead of exact depth

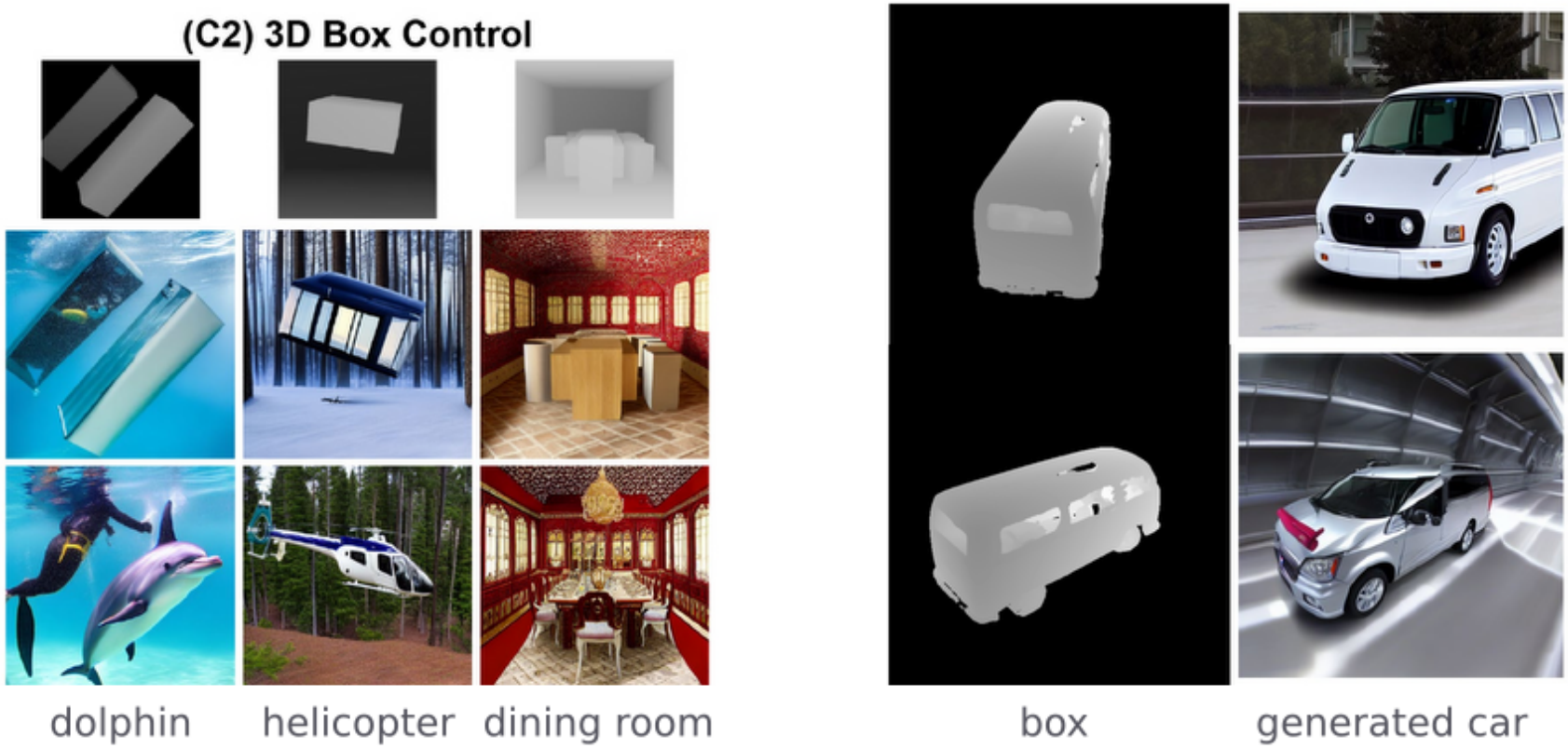


Figure 12: LooseControl's 3D box control (Bhat et al., 2024): three box layouts (top; "C2" is the paper's name for box control) and two images for each, for the prompts of an underwater photo of a dolphin and a snorkeler, a helicopter flying over a pine forest, and a royal dining room; and a box shaped like a van (white on black: depth) and two generated cars that fill it (figure by O. Dünkel).

De-Diffusion uses diffusion models to train VLMs

Vision-language models (VLMs) take an image as input and output text, for example a caption, or an answer to a question about the image. The best VLMs require industry-level datasets and compute: billions of image-text pairs and thousands of accelerators, so only companies can train them.

Text-to-image diffusion models also need large datasets and compute, but pretrained ones are available to the academic community. They already contain a great deal of knowledge about how words and images correspond. The key idea of De-Diffusion is to use this knowledge to train a VLM, with an autoencoder. The decoder of the autoencoder is the pretrained diffusion model, kept fixed; the encoder is the VLM, which turns an image into text; and training asks the decoder to redraw the image from that text. The name says that the encoder undoes the diffusion model.

De-Diffusion V1 (Wei et al., 2024) still required industry resources. De-Diffusion V2, developed at JHU and published as the Vision-Language-Vision (VLV) auto-encoder (Zhang et al., 2025), does not.

Images are represented by embeddings or quantized tokens

Any autoencoder represents an image by its latent, and the choice of latent decides what the representation is good for. Most deep networks represent an image by *continuous embeddings*: vectors of real numbers in the middle layer of an autoencoder, or the features learnt by self-supervised methods such as iBOT (image BERT pre-training with online tokenizer; Zhou et al., 2022), MaskFeat (masked feature prediction) and DiffMAE (diffusion masked autoencoder), which learn by predicting hidden parts of images. Others use *quantized tokens*: in a VQ-VAE (vector-quantized variational autoencoder; van den Oord et al., 2017) each feature vector of the encoder is replaced by the nearest of a fixed set of learnt vectors, the codebook, so the image becomes a grid of codebook indices, like words from a vocabulary. The first DALL-E and Google's Parti generate images by predicting such token sequences one token at a time. Both kinds of latent are unreadable for a person.

Images are represented by embeddings or quantized tokens

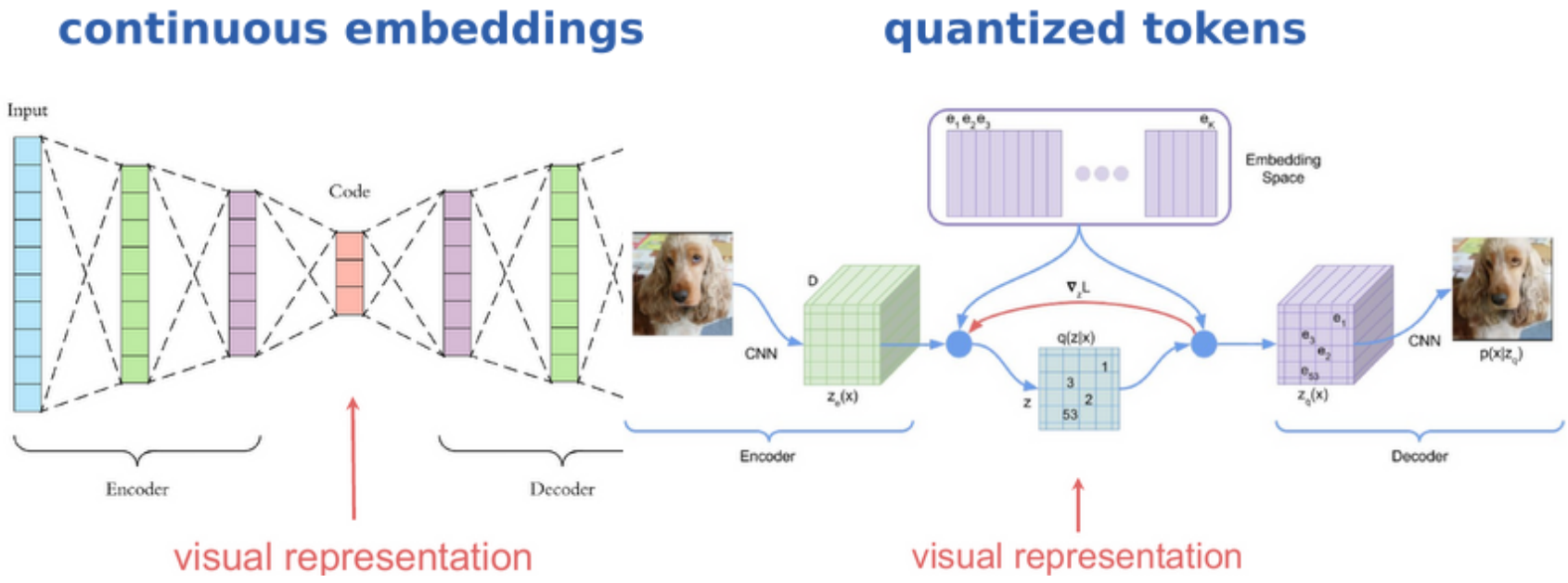


Figure 13: The two representations: an autoencoder whose middle layer (code) is the representation, and the VQ-VAE (van den Oord et al., 2017, Figure 1): encoder features $z_{\boldsymbol{e}}(x)$, the nearest codebook vectors (embedding space), and the decoder; the red arrow carries the decoder's gradient past the quantization to the encoder.

De-Diffusion represents an image as text

De-Diffusion asks whether an image can be represented as text: words that a person can read and that a language model can use. The encoder writes the image as a sequence of words; the decoder, a text-to-image model, draws the image back from them. If the redrawn image matches the original, the words have kept what the image shows, and the text is a visual representation. The figure is a sketch of the idea: its words are a tidied illustration (colored by image region), and the decoded picture stands for the redrawn image.



Figure 14: The idea on one image (De-Diffusion slides p. 5): a corgi with a hat and sunglasses on a city street is encoded into words ("a photograph closeup dog corgi wearing white hat corgi yellow glasses atop on red handle bicycle street nyc . . . "), and decoded back into an image of the same scene.

De-Diffusion text is a scrambled caption

The text that the trained encoder actually produces is a "scrambled caption": a list of the objects, attributes, relations and background of the image, with little grammar. For the corgi it names the dog and its breed, the hat, the sunglasses, the red handle and cart, the street, the people and cars behind it, the billboards and the city (nyc). The word order carries little meaning, and fragments such as "wht" and "harnehandle" appear: the encoder picks whatever tokens help the decoder most.



an attributing provided closeup dog corgi wearing wht yellow hat corgi and hat on glasses aboard an on red handle cart street nyc oscanadian corgi corgi presented description called an dog dog shown standing smiling incorporating wearing an wearing harnehandle fedora with smile yellow sunglasses yellow sunglasses and people and street wth foreground with people people among people cars multiple billboards cityscape and street billboards asphalt street cute screenshot street

Figure 15: De-Diffusion text of the corgi (De-Diffusion slides p. 12). Words of one color describe the same part of the image (blue: the dog; orange: the hat; yellow: the glasses; green: its pose and expression; red: the cart; light purple: the background); black words carry little content.

Humans encode many different modalities into text

Why text? People already use text to stand for other modalities. Speech is written down and read aloud (speech to text, text to speech); images get captions, and movies get scripts. In daily life, language often acts as an interface to higher-level cognition.

Why not ordinary captions? Captions written by people usually name only the most salient objects ("a dog wearing sunglasses") and leave out the rest of the scene, so they are not comprehensive enough to redraw the image. Why not deep embeddings? Text is interpretable, and it can be used directly by large language models (LLMs) and by text-to-image models, with no tuning of either; an embedding needs a model trained to read it.

Humans encode many different modalities into text

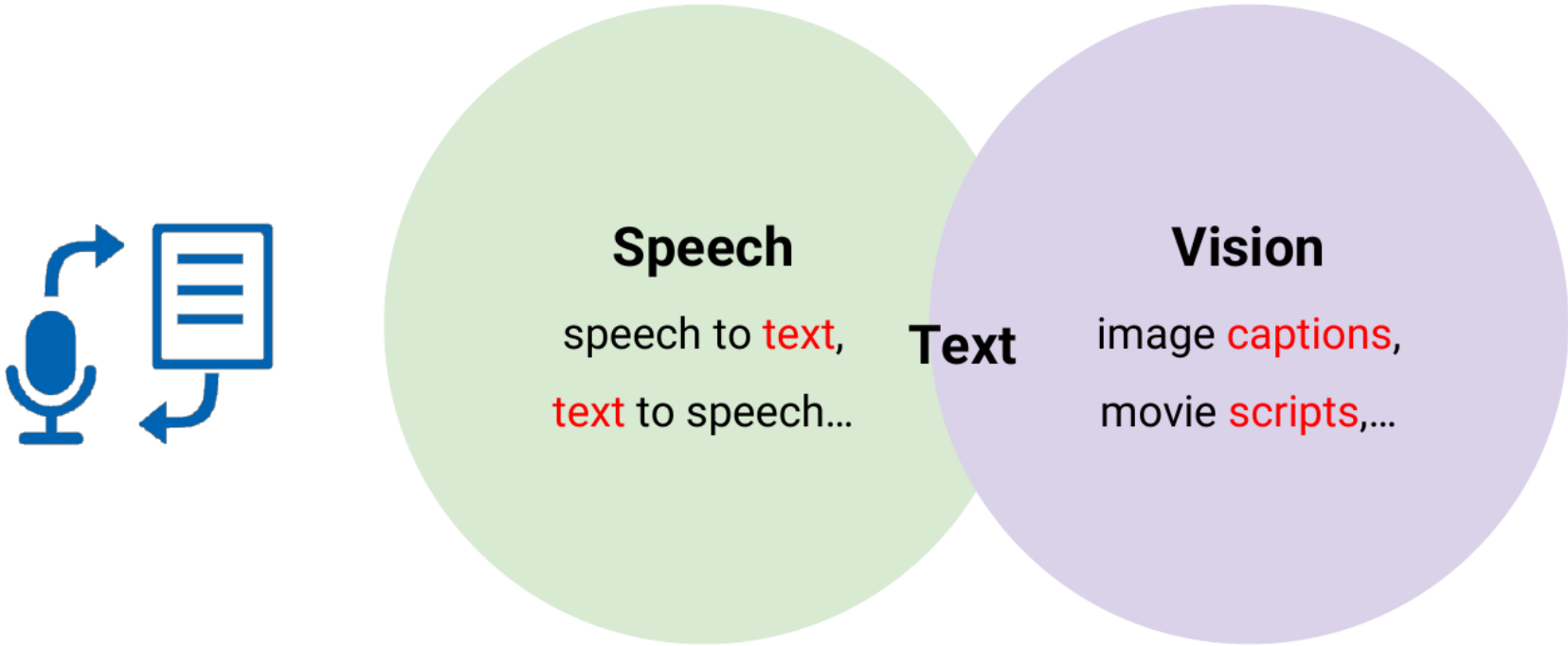


Figure 16: Text as the meeting point of speech and vision (De-Diffusion slides p. 14): speech and text are converted into each other; images are described by captions and films by scripts.

An autoencoder needs only images for training

An autoencoder has an encoder, which maps the input to latents, and a decoder, which maps the latents back to an output. The loss compares the output with the input, so the input is its own target: training only requires images, with no captions and no labels. The latents must carry whatever the decoder needs to rebuild the input.

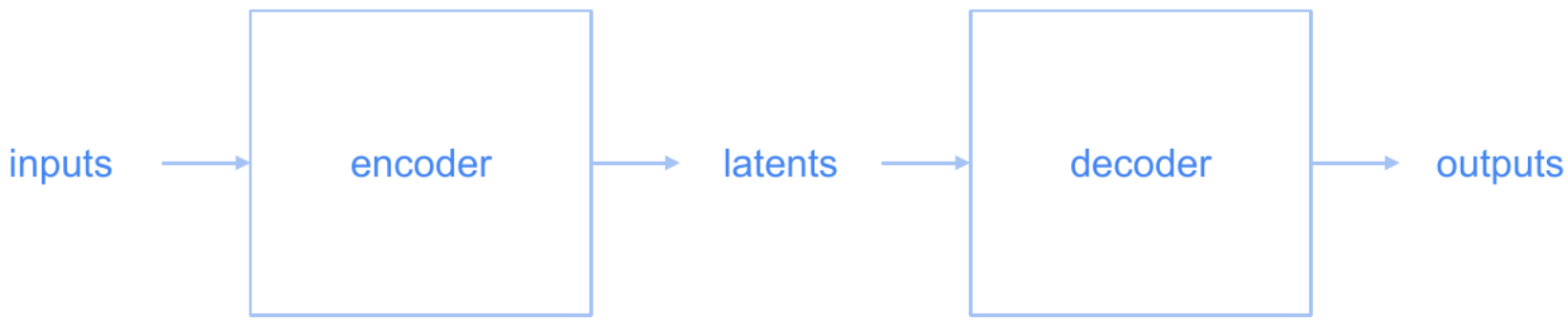


Figure 17: An autoencoder (De-Diffusion slides p. 18).

A pretrained text-to-image diffusion model is the decoder

De-Diffusion makes the latent text and the decoder a pretrained text-to-image diffusion model. The decoder is frozen (the closed lock); only the encoder E_ϕ (unrelated to the latent encoder $\mathcal{E}$ of Stable Diffusion), with parameters ϕ, is trained (the open lock). The loss is the decoder's own denoising loss: take a training image $\mathbf{x}_0$, noise it to $\mathbf{x}_t$, and ask the frozen network to predict the noise with the encoder's text as its prompt,

$$\min_{\phi} \mathbb{E}_{\mathbf{x_0},t,\varepsilon} \left\| \varepsilon - \varepsilon_\theta\left(\mathbf{x}_t, t, E_\phi(\mathbf{x}_0)\right) \right\|^2 \qquad (\theta \text{ frozen}).$$

The denoising loss is a per-pixel reconstruction loss: its gradient flows through the frozen decoder into the encoder and changes the text so that the decoder can redraw the image better. To minimize the error the text has to be accurate and comprehensive, since anything it leaves out the decoder can only guess. Training thereby unpacks the knowledge encapsulated within the text-to-image model, which already knows what "corgi" looks like. (The paper's decoder predicts $\mathbf{v} = \sqrt{\bar{\alpha}_t}\,\varepsilon - \sqrt{1-\bar{\alpha}_t}\,\mathbf{x}_0$ in place of ε; the idea is the same.)

A pretrained text-to-image diffusion model is the decoder

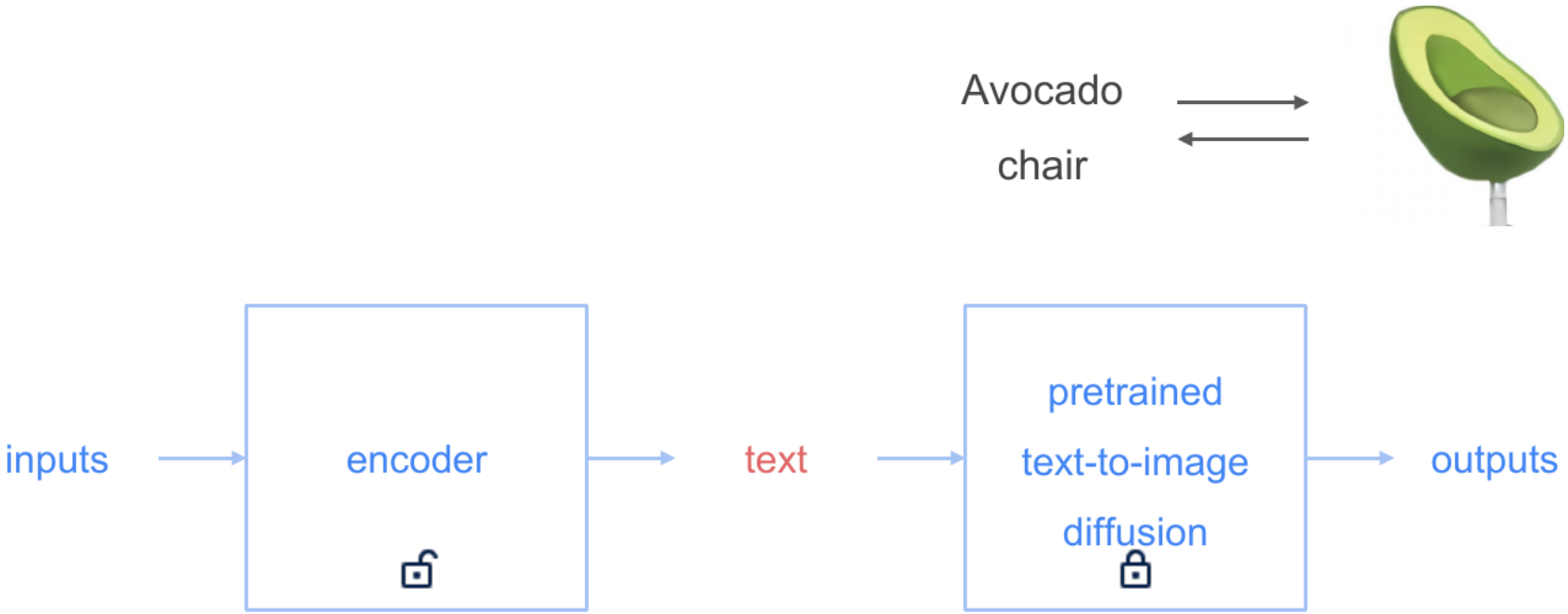


Figure 18: Diffusion as the decoder (De-Diffusion slides p. 20): the encoder (open lock: trained) writes text, the pretrained text-to-image diffusion model (closed lock: frozen) redraws the image. The words "avocado chair" and the chair drawn from them stand for the knowledge of words and images in the decoder.

Discrete text tokens are trained with Gumbel softmax

The text is discrete: at each of its positions the encoder must choose one word from a vocabulary. Choosing, an arg max, has no useful gradient, so the reconstruction loss cannot tell the encoder how to change its choice. De-Diffusion uses the Gumbel softmax (Jang et al., 2017), a smooth stand-in for a random choice. Let π_i be the encoder's probability of word i, and draw independent Gumbel noise g_i; the soft word is the vector $\mathbf{s}$ with entries

$$s_i = \frac{\exp\left((\log \pi_i + g_i)/\lambda\right)}{\sum_j \exp\left((\log \pi_j + g_j)/\lambda\right)}, \qquad g_i = -\log(-\log u_i), \quad u_i \sim \mathcal{U}(0,1).$$

$\mathbf{s}$ is a vector of weights over the vocabulary, a soft one-hot vector, and the decoder receives the corresponding weighted average of the word embeddings. Gradients pass through $\mathbf{s}$ to the probabilities π_i. The temperature λ (written τ by Jang et al.; here τ is the text encoder) controls how hard the choice is: as $\lambda \to 0$, $\mathbf{s}$ approaches the one-hot vector of a single word. De-Diffusion anneals λ from 2.0 to 0.3.

Discrete text tokens are trained with Gumbel softmax

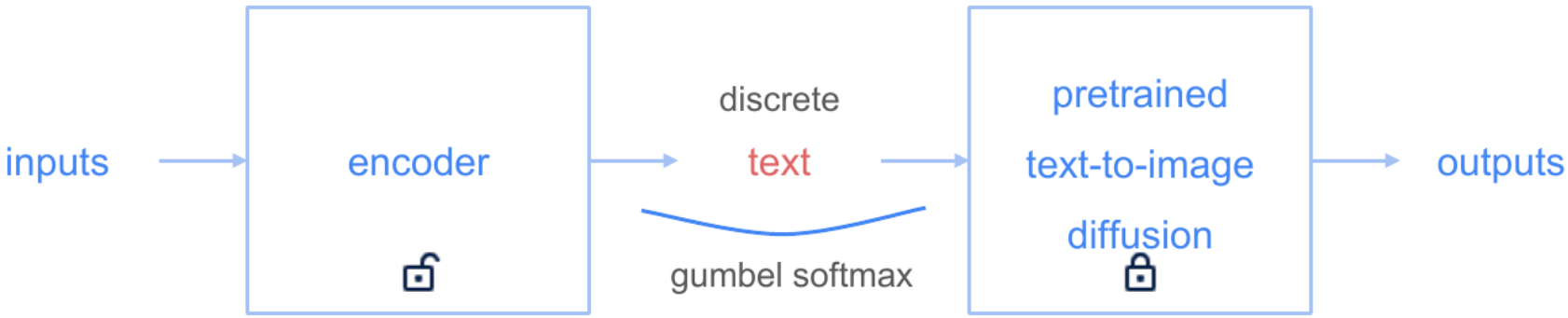


Figure 19: The full picture (De-Diffusion slides p. 22): discrete text between the encoder and the frozen decoder, trained with the Gumbel softmax.

The Gumbel-max trick samples a word

Why Gumbel noise? Take the hard choice $i^* = \arg\max_i (\log \pi_i + g_i)$ with g_i independent Gumbel variables. Then $P(i^* = k) = \pi_k$: adding Gumbel noise to the log probabilities and taking the largest draws a word from the distribution π exactly (the Gumbel-max trick).

The reason is a property of the Gumbel distribution, whose cumulative distribution function is $F(g) = \exp(-e^{-g})$. The event that word k wins is the event that $\log \pi_k + g_k \geq \log \pi_j + g_j$ for all j. Given g_k, the other words lose independently, with probability $F(\log \pi_k + g_k - \log \pi_j) = \exp(-e^{-g_k} \pi_j / \pi_k)$ each. Multiplying over $j \neq k$ and integrating over g_k against its density $e^{-g_k} \exp(-e^{-g_k})$ gives

$$P(i^* = k) = \int e^{-g} \exp\Big(- e^{-g} \sum_j \frac{\pi_j}{\pi_k} \Big)\, dg = \frac{\pi_k}{\sum_j \pi_j} = \pi_k .$$

The Gumbel softmax replaces the arg max by a softmax at temperature λ, which is differentiable; as $\lambda \to 0$ it becomes the one-hot vector of i^*, so a low final temperature makes the training text close to the text used at test time.

The encoder is an image backbone with an attentional pooler

The encoder has two parts. The image backbone is a pretrained CoCa (Contrastive Captioner) ViT-L, a large vision transformer, at 288×288; it is frozen by default (the closed lock), and the paper also trains one from scratch (the open lock). The attentional pooler is five transformer blocks: 75 learnt query vectors cross-attend to the image features and to each other, and a linear layer maps each query to scores over the vocabulary. Each query gives one word, at a fixed position. The vocabulary is the 49K tokens of CLIP's text encoder, without punctuation; 75 words plus a start and an end token fill the 77-token input of that encoder.

The decoder is an Imagen-style (Saharia et al., 2022) diffusion U-Net of 600M parameters that generates 64×64 images, with a CLIP text encoder (OpenCLIP ViT-H/14), pretrained on WebLI (Google's web collection of images with text) and frozen. OpenCLIP is an open-source reproduction of CLIP; "Imagen-style" means a diffusion model on pixels, at 64×64, like Google's Imagen. De-Diffusion itself is trained on WebLI images alone, for 500K steps with batches of 2048: about 2.5 days on 64 TPU chips (tensor processing units, version 4): industry-scale resources.

The encoder is an image backbone with an attentional pooler

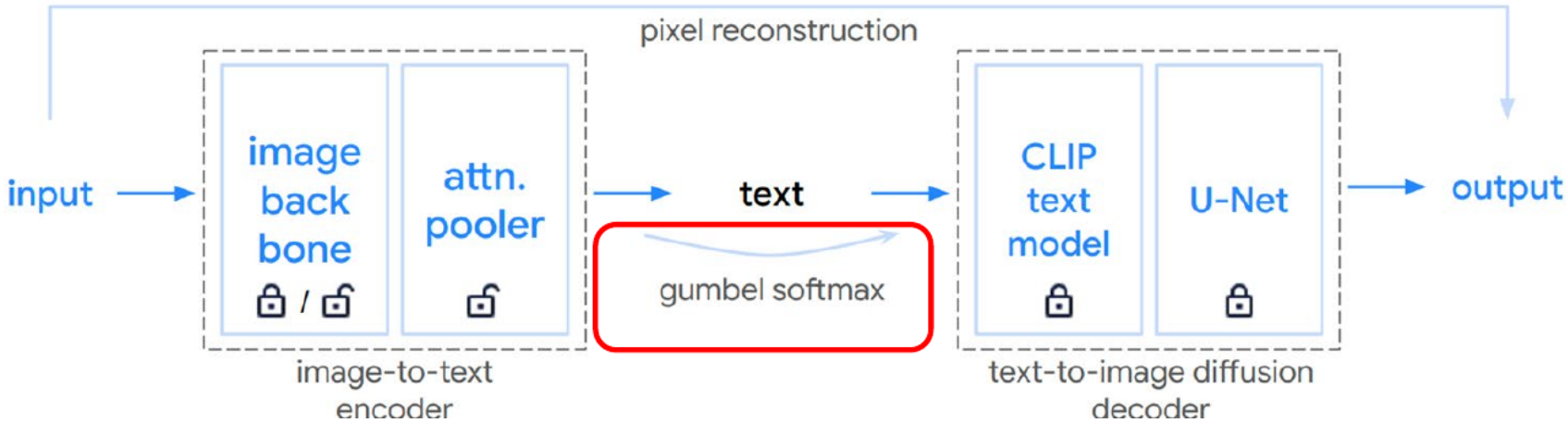


Figure 20: The architecture (De-Diffusion slides p. 42): image backbone and attentional pooler (the image-to-text encoder), the text with the Gumbel softmax (red box), the CLIP text model and U-Net (the text-to-image diffusion decoder), and the pixel reconstruction loss.

De-Diffusion text is a transferable text-to-image prompt

Once trained, the encoder turns any image into text, and the text can be given to text-to-image tools other than the decoder it was trained with. For a photograph of a glass jar filled with orange slices and berries against a pink wall, the text reads, in part, "... closeup berries jar through large ... glass jar ... glass on an on peach hardwood closeup glass homemade mixed glass jar ... oranges fruit shown slices ... black grapes ... black blueberry ... pink closeup facing that background pink wall ...". Imagen, Midjourney and Stable Diffusion XL (Podell et al., 2024), given this text, each redraw a jar of orange slices and dark berries in front of a pink wall.



Figure 21: The original photograph and the images that Imagen, Midjourney and Stable Diffusion XL generate from its De-Diffusion text (De-Diffusion slides p. 23).

Human captions describe only the most salient content

A caption written by a person shows what the comprehensiveness buys. The COCO caption of the jar photograph is "A jar filled with different types of fruit on a table." Prompted with it, the three tools draw a jar of mixed fruit (kiwi, strawberries, grapes) on a table, with different backgrounds and lighting. The orange slices, the berries, the pink wall and the close-up view are lost, because the caption never mentions them.

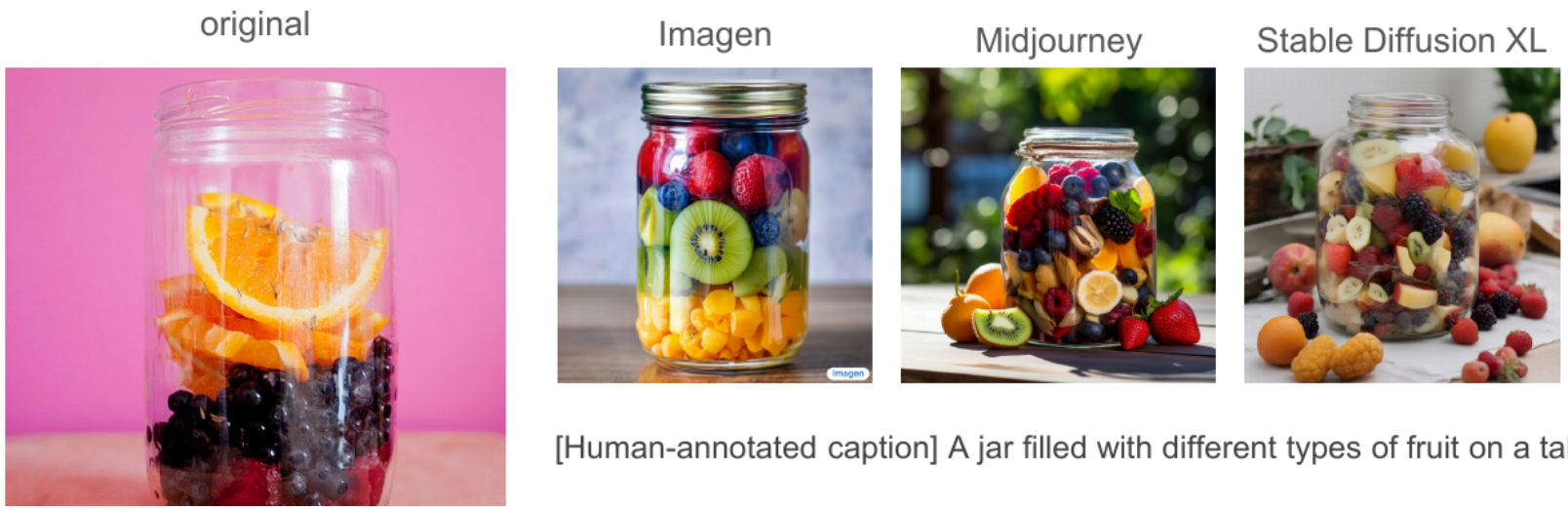


Figure 22: The same tools prompted with the human caption (De-Diffusion slides p. 24).

De-Diffusion text reconstructs better than captions

The comparison can be measured. Stable Diffusion v2, a third-party model, redraws the 30K images of the MS-COCO (Microsoft Common Objects in Context) 2014 validation set from four kinds of text: De-Diffusion text, the human captions of COCO, and the captions of two VLMs, BLIP-2 (Bootstrapping Language-Image Pre-training) and PaLI-X (Pathways Language and Image model). The redrawn images are compared with the real ones by the FID, Fréchet inception distance (Heusel et al., 2017): both sets of images, real (r) and generated (g), are passed through Inception-v3, an image classifier trained on ImageNet, a Gaussian is fitted to each set of features, and the FID is the Fréchet distance between the two Gaussians, $\|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \mathrm{tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$. Lower is better.

The FID is computed at each guidance scale w from 1.5 to 8. From $w = 2$ on, De-Diffusion text gives the lowest FID; its best value, 6.43 at $w = 3$, is well below the best of the human captions (8.08, also at $w = 3$), BLIP-2 (8.46) and PaLI-X (8.50). For all four texts the FID grows at large w, where guidance trades diversity for sharpness.

De-Diffusion text reconstructs better than captions

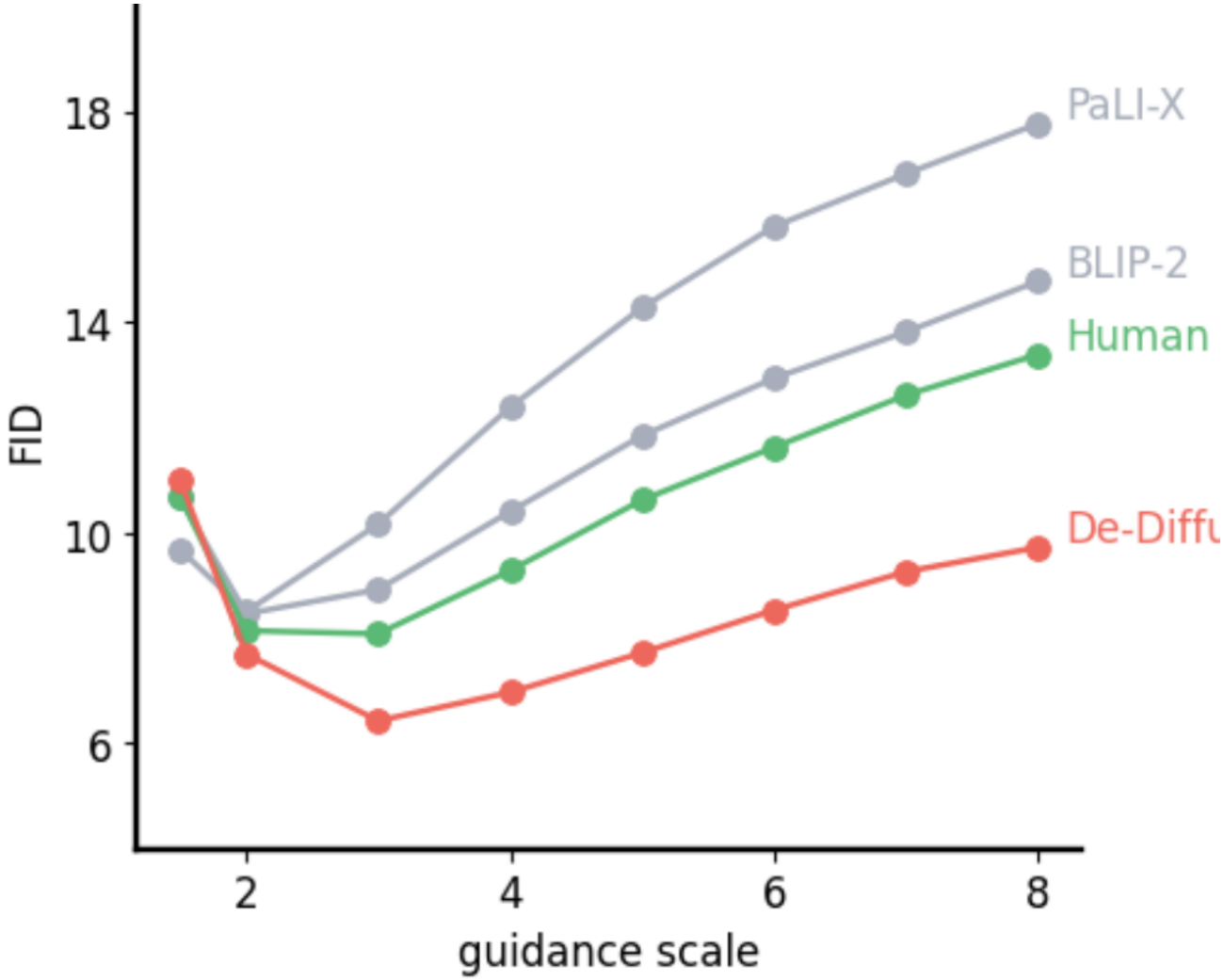


Figure 23: FID of images redrawn by Stable Diffusion v2 from four kinds of text, against the guidance scale (De-Diffusion slides p. 25): PaLI-X and BLIP-2 captions (grey), human captions (green), De-Diffusion text (red).

De-Diffusion text also prompts for synthetic images

The encoder was trained on photographs, but it also describes images that are themselves generated, such as digital art. For a colorful painting of a woman's face covered in flowers, paint and confetti, its text names the style as well as the content: "an art . . . digitally . . . art illustration woman face wearing colorful colorful paints face painted head pink lipstick . . . colourful confetti . . . realism pin up . . . teal background". The three tools redraw a face in the same style.

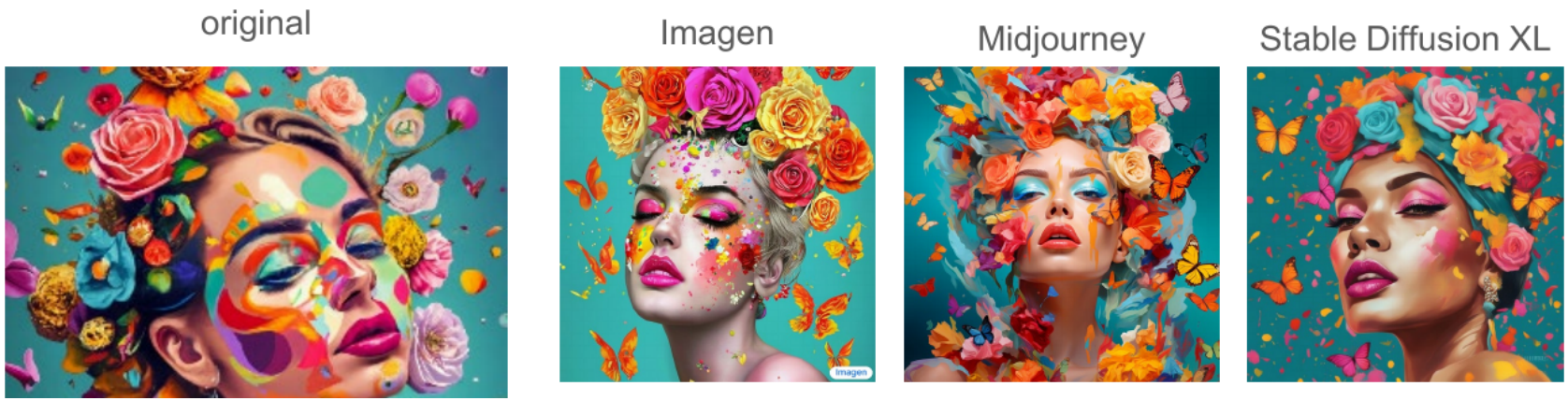


Figure 24: A generated picture and the images that three tools draw from its De-Diffusion text (De-Diffusion slides p. 26).

An LLM answers visual questions from De-Diffusion text

Text can also be read by a language model that has never seen an image. The prompt of the LLM holds an instruction ("Answer the question given the context."), the De-Diffusion text of the image as context, and a question. For a photograph of a red and yellow railcar at a platform, the text contains "train train", "metrotram in a red livery", "wearing a yellow", "white stripes markings" and "yellow yellow stripes train". Asked "What other big vehicle is often painted about the same shade as this vehicle?", PaLM 2, a text-only LLM (Anil et al., 2023), answers "Fire truck": the text supplies the color, and the LLM supplies the knowledge that fire trucks are red. This task is visual question answering (VQA), and this question needs knowledge outside the image.

An LLM answers visual questions from De-Diffusion text



[Prompt] Answer the question given the context.

a colcandidenver lantic closeup former recent **train train** parked tradition enclosed **metrotram in a red livery** it on railroad platform containing **wearing a yellow pol** surround a knob beside platform near a under platform shelter right there and roof shadows and platform and tracks etc wore worn worn mau maroon brown **white stripes markings** content worn **yellow yellow stripes train** pretoria namibia railway platform train operator worn brown windows platform platform

Question: What other big vehicle is often painted about the same shade as this vehicle?

[LLM PaLM 2] Fire truck.

Figure 25: Visual question answering with an LLM (De-Diffusion slides p. 30); the colored words are those that describe the vehicle.

Text makes multimodal few-shot learning modular

Flamingo (Alayrac et al., 2022) is a VLM trained on web pages where images and text are interleaved. Given a few examples in its prompt (images with their answers) it answers for a new image: few-shot learning. De-Diffusion gets the same ability without any joint training: each image of the prompt is replaced by its De-Diffusion text, and the prompt goes to PaLM 2-L. The table compares them on VQAv2 (accuracy, %), OKVQA (outside-knowledge VQA, accuracy, %) and COCO captioning (CIDEr, Consensus-based Image Description Evaluation: agreement with five human captions), with 0 and 32 examples in the prompt.

	VQAv2 0 / 32 shots	OKVQA 0 / 32 shots	COCO caption 0 / 32 shots
Flamingo-80B	56.3 / 67.6	50.6 / 57.8	84.3 / **113.8**
De-Diffusion + PaLM 2-L	**67.2** / **68.4**	**57.0** / **60.6**	**88.5** / 103.7

De-Diffusion wins every column except 32-shot captioning. Its gains are easy exploitation of advanced LLMs (just inference) and easy data collection (just images, no interleaved data); its encoder trains 135M parameters, Flamingo 10B.

Text makes multimodal few-shot learning modular

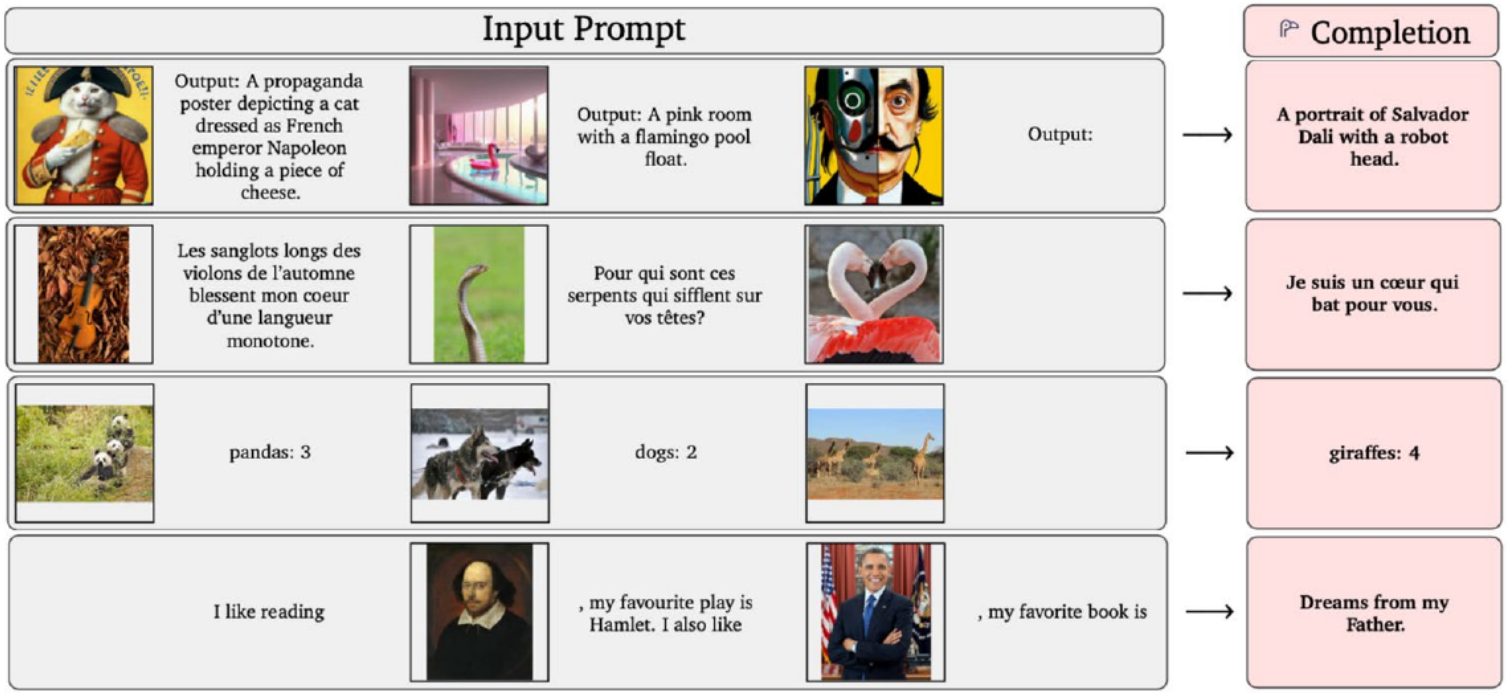


Figure 26: Flamingo's prompts (Alayrac et al., 2022, Figure 1; from the De-Diffusion slides p. 31): each row interleaves example images with their outputs and ends with a new image; the pink box ending each row is the completion Flamingo writes.

A text-only chatbot discusses the image through its text

ChatGPT, used here as a text-only chatbot, is given the corgi's De-Diffusion text as keywords and asked "What is unusual about this image?" It answers that it is unusual to see a corgi riding a bicycle, especially in a busy city like New York. The text says "red handle cart"; ChatGPT reads it as a bicycle and combines it with the other words (corgi, street, nyc) and common sense about dogs.



C

Given an image with keywords:

"an attributing provided closeup dog corgi wearing wht yellow hat corgi and hat on glasses aboard an on red handle cart street nyc oscanadian corgi corgi presented description called an dog dog shown standing smiling incorporating wearing an wearing harnehandle fedora with smile yellow sunglasses yellow sunglasses and people and street wth foreground with people people among people cars multiple billboards cityscape and street billboards asphalt street cute screenshot street"

What is unusual about this image?

It is unusual to see a corgi riding a bicycle, especially in a busy city like New York…

Figure 27: A dialogue with ChatGPT about the corgi through its De-Diffusion text (De-Diffusion slides p. 33).

More examples of De-Diffusion text

For a photograph of a yellow bus seen in a round mirror the text names “yellow bus”, “mirror”, “through a mirror” and “reflection”. A steampunk drawing gives “robotic cat” and “steampunk cyborg cat”. A lion logo gives “lions”, “silhouette” and “line symbol”. For the Grand Canyon it names the place: factual knowledge.



photo



creative art



abstract symbol



knowledge

Figure 28: The four pictures of the examples (De-Diffusion slides pp. 34, 35, 36 and 38).

Continuous embeddings make training easy and fast

Can De-Diffusion be made better, with less compute and data? Its discrete text, trained with the Gumbel softmax, is hard to train and slow to converge; the VLV paper reports that the discrete tokens cause training inefficiency and lose fine-grained detail. De-Diffusion V2, by Junfei Xiao, Tiezheng Zhang and colleagues at JHU, published as the VLV auto-encoder (Zhang et al., 2025), replaces the words by continuous embeddings.

Its encoder (initialized from Florence-2, a Microsoft vision model) produces 77 vectors, one per learnt query, which fill the text encoder's whole 77-token input, projected by a small multilayer perceptron (MLP) into the input space of the text encoder of the frozen decoder, Stable Diffusion 2.1. The vectors play the role of the words: the decoder reads them as if they were a prompt. Training is the usual denoising loss on 40M images with no text. The bottleneck is still there, because the frozen decoder only understands vectors that resemble text embeddings: the regularization comes from freezing the decoder.

Continuous embeddings make training easy and fast

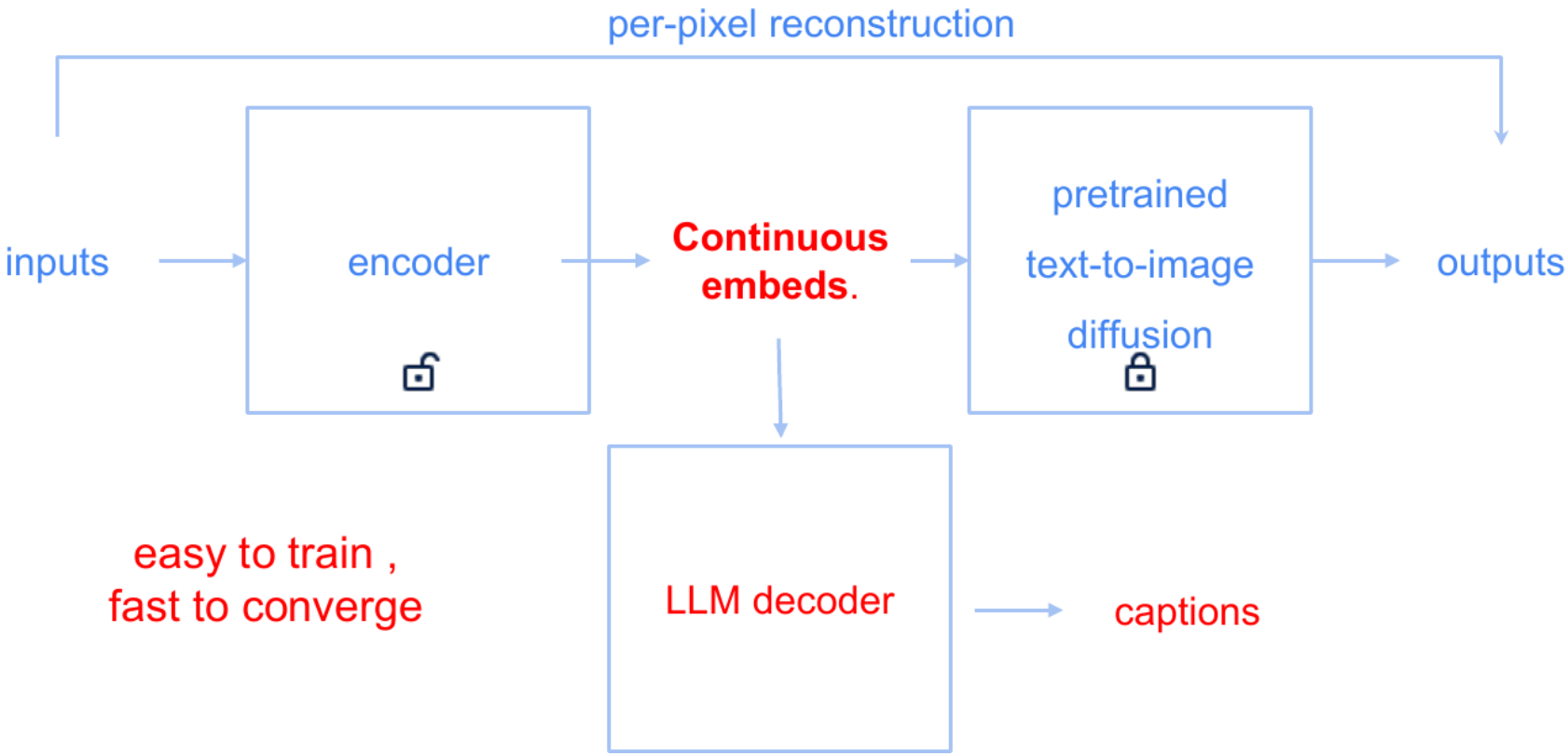


Figure 29: From discrete to continuous (De-Diffusion slides p. 43): continuous embeddings between the encoder and the frozen diffusion decoder (stage 1; the loss is drawn as per-pixel, as in V1, and is the denoising loss on Stable Diffusion's latent), and an LLM decoder that turns them into captions (stage 2, trained afterwards).

An LLM decoder writes readable captions of any length

Continuous embeddings cannot be read by a person. The second stage trains a decoder of captions: the 77 vectors pass through the frozen text encoder, a small MLP maps the result into the input space of a pretrained LLM, Qwen2.5 (Qwen Team, 2024), and the LLM is fine-tuned to write a caption from it. The training captions are about 6M detailed descriptions written by Gemini 2.0 Flash, a commercial multimodal model of Google. An LLM has no limit on the length of its output and writes readable text, so the captions are long, grammatical and detailed.

The whole training costs under 1,000 US dollars, less than 1,000 GPU (graphics processing unit) hours, against the 64 TPU chips for 2.5 days of De-Diffusion V1, and it uses 40M images, against the billions of WebLI.

An LLM decoder writes readable captions of any length

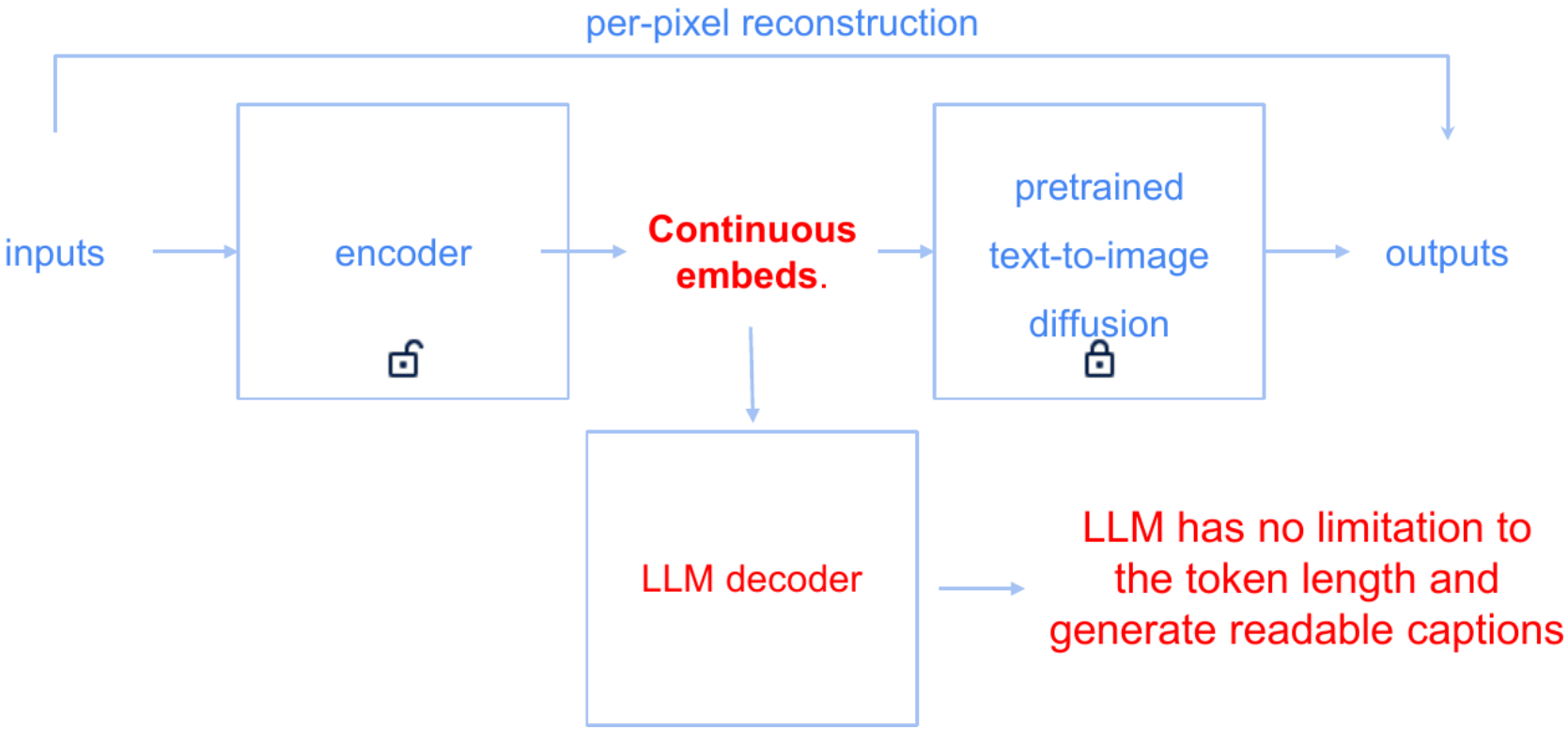


Figure 30: The LLM decoder (De-Diffusion slides p. 45).

The captions are detailed enough to redraw the image

For a photograph of a brown bear in grass, the autoencoder's own reconstruction, drawn by the diffusion decoder from the continuous embeddings, keeps the content (a bear's face in grass, the same pose) but darkens the fur. The caption of V2's captioner (with a 1.5B-parameter LLM) begins "A close-up shot of a brown bear in a grassy field. The bear is looking directly at the camera with a slightly open mouth, revealing a pink tongue sticking out. The bear's fur is a mix of light brown and dark brown hues . . .". GPT-4o's caption (OpenAI's commercial multimodal model) begins "A large, fluffy bear sits calmly on a grassy field, its thick fur a mix of golden and brown shades . . .". On COCO, the FID of images redrawn from V2's captions is close to that of GPT-4o's (6.64 against 6.20 at guidance 2), and in the paper's rating of caption quality on a 0–6 scale (by people and by a VLM judge) the two score almost equally (5.18 against 5.23).

The captions are detailed enough to redraw the image

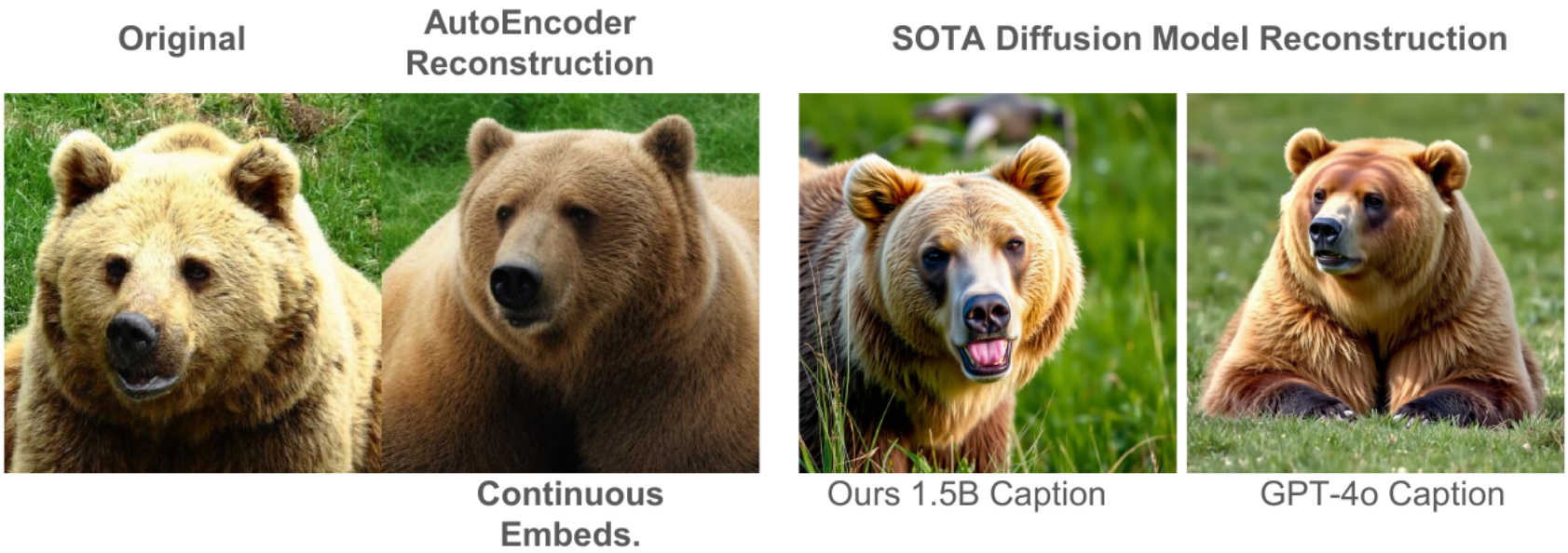


Figure 31: Original, autoencoder reconstruction, and images redrawn from V2's caption and GPT-4o's (De-Diffusion slides p. 47).

V2 captions are readable; De-Diffusion text is scrambled

The V2 caption ("Latest") is readable, though not always right (it calls the cat white and lying); De-Diffusion text ("Baseline") repeats words.



Latest: **A tan dog** with **a black nose** nudges a white cat with a black striped tail. The cat is lying down with its paws outstretched. **The dog is looking down at the cat with a gentle expression.** The background is out of focus, but it appears to be a paved surface. The lighting is soft and natural. The dogs fur is short and shaggy. The cat is looking slightly away from the camera. **The dog's nose is very close to the cat's face.** The dog's head is tilted slightly. The dog's eyes are dark brown. The cat's fur is long and fluffy. The dog's ears are perked up. The dog's whiskers are long and white. The cat's tail is thick.

Baseline: a col special awesome generic sphoto dog an shepherd dog sitting **smelling smelling** an cat showing a whites eachother it looking eachother there ently relating a a cat polcat a cat there blurry towards a cat **cat cat** among **subtle subtle** sunny shadows snow road sunny background parenparencat cat redness **browns brown** and ently feline parenvintage **regard regard** navarshepherd optimized macro sensual canine dog close vintage closeup closeup beak kiss

Figure 32: V2 caption (green: details marked correct) and De-Diffusion text (red: repeats); De-Diffusion slides p. 49.

Summary

Diffusion models add Gaussian noise step by step and train a network to predict that noise; the trained network estimates the posterior mean of the clean image.

Generation starts from pure noise; each step moves a little towards the current guess of the clean image, which is noisy gradient ascent on the log density of the noisy images, annealed from coarse to fine.

Text-conditioned diffusion: Stable Diffusion denoises a latent; cross-attention brings in the prompt, and classifier-free guidance samples from $p(\mathbf{z})\, p(y \mid \mathbf{z})^w$.

Image-conditioned diffusion: ControlNet adds edges, poses or depth through a trainable copy and zero convolutions; the prompt and the control add as likelihood terms.

De-Diffusion: an autoencoder with a frozen text-to-image decoder learns to write an image as text; its text prompts other text-to-image tools and lets text-only LLMs answer visual questions.

De-Diffusion V2 uses continuous embeddings and an LLM decoder for readable captions, at a small fraction of the cost.

References

[1] W. Ma, Q. Liu, J. Wang et al. Generating images with 3D annotations using diffusion models. *ICLR*, 2024. Course slides by W. Ma (3D-DST slides).

[2] C. Wei, C. Liu, S. Qiao, Z. Zhang, A. Yuille and J. Yu. De-Diffusion makes text a strong cross-modal interface. *CVPR*, 13492–13503, 2024. Course slides by A. Yuille, C. Wei and J. Xiao (De-Diffusion slides).

[3] T. Zhang, Y. Li, Y.-C. Chou, J. Chen, A. Yuille, C. Wei and J. Xiao. Vision-language-vision auto-encoder: scalable knowledge distillation from diffusion models. *NeurIPS*, 2025.

[4] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *ICML*, 2256–2265, 2015.

[5] J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 6840–6851, 2020.

[6] O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. *MICCAI*, 234–241, 2015.

[7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer. High-resolution image synthesis with latent diffusion models. *CVPR*, 10684–10695, 2022.

References

[8] A. Radford, J. W. Kim, C. Hallacy et al. Learning transferable visual models from natural language supervision. *ICML*, 8748–8763, 2021.

[9] A. Vaswani, N. Shazeer, N. Parmar et al. Attention is all you need. *NeurIPS*, 5998–6008, 2017.

[10] J. Ho and T. Salimans. Classifier-free diffusion guidance. *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.

[11] D. Podell, Z. English, K. Lacey et al. SDXL: improving latent diffusion models for high-resolution image synthesis. *ICLR*, 2024.

[12] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch and D. Cohen-Or. Prompt-to-prompt image editing with cross-attention control. *ICLR*, 2023.

[13] L. Zhang, A. Rao and M. Agrawala. Adding conditional control to text-to-image diffusion models. *ICCV*, 3836–3847, 2023.

[14] S. F. Bhat, N. J. Mitra and P. Wonka. LooseControl: lifting ControlNet for generalized depth conditioning. *SIGGRAPH Conference Papers*, 2024.

References

[15] A. van den Oord, O. Vinyals and K. Kavukcuoglu. Neural discrete representation learning. *NeurIPS*, 6306–6315, 2017.

[16] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. Yuille and T. Kong. iBOT: image BERT pre-training with online tokenizer. *ICLR*, 2022.

[17] E. Jang, S. Gu and B. Poole. Categorical reparameterization with Gumbel-softmax. *ICLR*, 2017.

[18] C. Saharia, W. Chan, S. Saxena et al. Photorealistic text-to-image diffusion models with deep language understanding. *NeurIPS*, 36479–36494, 2022.

[19] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *NeurIPS*, 6626–6637, 2017.

[20] J.-B. Alayrac, J. Donahue, P. Luc et al. Flamingo: a visual language model for few-shot learning. *NeurIPS*, 23716–23736, 2022.

[21] R. Anil, A. M. Dai, O. Firat et al. PaLM 2 technical report. arXiv:2305.10403, 2023.

[22] Qwen Team, A. Yang, B. Yang et al. Qwen2.5 technical report. arXiv:2412.15115, 2024.